



SEGi
University &
Colleges



College Name	University of Hertfordshire SEGi College Subang Jaya											
Programme Name	BACHELOR OF SCIENCE (HONS) COMPUTER SCIENCE (ARTIFICIAL INTELLIGENCE)											
Module Name	Artificial Intelligence Project	Module Code	6COM2017									
		Semester	March 2026									
Module Leader	Ms Anizah Abu Bakar	Assessment Type	Final Report									
Lecturer Name	Mr. Abdul Arriff Abdul Jalal											
Student's declaration	I hereby certify that this assignment is my own work and where materials have been used from other resources, they have been properly acknowledged. I also understand I will face the possibility of failing the module if the content of this assignment is plagiarized. <table><tr><th>No.</th><th>Name</th><th>Student ID</th><th>Signature</th></tr><tr><td>1</td><td>Kenneth Chong Vui Kin</td><td>SCSJ2100435</td><td></td></tr></table>				No.	Name	Student ID	Signature	1	Kenneth Chong Vui Kin	SCSJ2100435	
No.	Name	Student ID	Signature									
1	Kenneth Chong Vui Kin	SCSJ2100435										
Release Date		Submission Due Date	12/06/2026	Marks Obtained: 								
Date Received		Student's work assessed by / date										

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Mr Abdul Arriff Abdul Jalal for his invaluable guidance, continuous support, constructive feedback, and encouragement throughout the completion of this Final Year Project. His expertise and dedication have greatly contributed to the success of this work.

I would also like to extend my heartfelt appreciation to my family for their unwavering support, patience, understanding, and motivation throughout my studies. Their encouragement has been a constant source of strength and inspiration during challenging times.

Finally, I would like to thank all lecturers, friends, and classmates who have provided assistance, shared knowledge, and offered encouragement during the course of this project. Their support is greatly appreciated.

ABSTRACT

User interface (UI) quality is essential for ensuring positive user experiences, yet traditional usability evaluation methods are often time-consuming, subjective, and dependent on expert knowledge. This project presents the development of an AI Chatbot for UI Improvement, a web-based system designed to automate the identification of UI/UX issues through screenshot analysis and conversational feedback.

The proposed system combines OpenCV-based image processing, rule-based usability evaluation, and Google Gemini 2.5 Flash to provide comprehensive UI assessments. Uploaded screenshots are analysed against usability principles derived from WCAG 2.2 accessibility standards and Nielsen's heuristics. The extracted metrics and audit findings are then used to guide the AI model in generating structured recommendations for improving usability, accessibility, and interface design quality.

The system was developed using FastAPI, MongoDB Atlas, HTML, CSS, and JavaScript, and evaluated using multiple UI screenshots from web and mobile applications. The results show that the system successfully detected common usability issues such as insufficient colour contrast, small interactive elements, and visual clutter while providing relevant improvement suggestions. The proposed hybrid approach demonstrates the potential of combining rule-based auditing with AI-powered reasoning to support faster and more consistent UI/UX evaluation.

Table of Content

Table of Content	3
List of Figures	6
List of Tables	7
1. Introduction	8
1.1. Introduction.....	8
1.2. Problem Statement.....	10
1.3. Objectives.....	11
1.4. Research Questions.....	12
1.5. Solution Proposed	14
2. Literature Review	16
2.1. Literature Review and Options for Problem Solving.....	16
2.2. Preferred Option and Justifications	19
3. Methodology	21
3.1. Methodology Used.....	21
3.2. Use Case Diagram	23
3.3. System Architecture Design	24
3.4. System Flowchart.....	29
3.5. Image Processing and Feature Extraction.....	31
3.6. Rule-Based Usability Evaluation	31
3.7. UI Quality Scoring Mechanism	32
3.8. AI-Powered Feedback Generation	33
3.9. Prompt Engineering Strategy	33
3.10. Database.....	34
4. Implementation	35
4.1. System Architecture Overview	35
4.2. Frontend Implementation	36
4.2.1. Chat Interface Design	36
4.2.2. Asynchronous Communication.....	37
4.2.3. Conversational Interface Features	37
4.3. FastAPI Backend Implementation	38
4.3.1. API Initialization.....	38

4.3.2.	API Endpoint Implementation	39
4.3.3.	Request Validation	39
4.3.4.	Asynchronous Processing	39
4.3.5.	Error Handling and Retry Mechanism	40
4.4.	OpenCV Computer Vision Implementation	41
4.4.1.	Image Preprocessing	41
4.4.2.	Whitespace Analysis	41
4.4.3.	Touch Target Analysis	42
4.4.4.	Contrast Ratio Analysis	42
4.4.5.	Visual Debugging Output	42
4.5.	Rule-Based Expert System	42
4.5.1.	WCAG Accessibility Rules	42
4.5.2.	Nielsen's Heuristics Integration	43
4.5.3.	UI Quality Classification	43
4.5.4.	Severity Classification	44
4.5.5.	Rule-to-AI Prompt Bridge	44
4.6.	Prompt Engineering and Gemini AI Integration	45
4.6.1.	System Instruction Design	45
4.6.2.	Dynamic Prompt Construction	45
4.6.3.	Structured Response Formatting	45
4.6.4.	Conversational Memory	46
4.7.	MongoDB Database Implementation	46
4.7.1.	Database Connectivity	46
4.7.2.	Audit Record Storage	46
4.7.3.	Asynchronous Database Operations	47
4.8.	Overall System Workflow	47
5.	Findings and Results	48
5.1.	Experimental Setup and Parameters	48
5.2.	Technical Metrics Analysis	49
5.3.	Backend Analysis Output	50
5.4.	Rule-Based Usability Detection Results	51
5.5.	Severity Distribution Analysis	53
5.6.	Usability Issue Category Analysis	54
5.7.	AI-generated Feedback Evaluation	55

5.8.	System Performance Evaluation	57
5.9.	Functional Testing Results	58
5.10.	Discussion of Findings	59
6.	Limitations, Future Enhancement, Risk Register, Discussion and Conclusion	60
6.1.	System Limitations.....	60
6.1.1.	Technical Limitations	60
6.1.2.	AI and Prompt Engineering Limitations	61
6.1.3.	Evaluation Limitations	61
6.2.	Future Enhancement	62
6.2.1.	AI and Analysis Improvements.....	62
6.2.2.	System Enhancements	63
6.2.3.	Feature Expansions.....	64
6.3.	Risk Register.....	64
6.3.1.	Technical Risks	64
6.3.2.	Security and Privacy Risk	65
6.3.3.	Ethical Concerns	65
6.4.	Discussion of Results.....	66
6.4.1.	Hybrid System Effectiveness.....	66
6.4.2.	Rule-based consistency	66
6.4.3.	AI-Powered Recommendation Quality	66
6.4.4.	System Performance	66
6.5.	Conclusion	67
7.	References.....	68
8.	Appendices.....	72
8.1.	Gantt Chart	72
8.2.	User Test Case	73
8.3.	Code Implementation	83
8.4.	Proposal Report	111

List of Figures

Figure 3.1.1 Methodology Used: Agile.....	21
Figure 3.2.1 Use Case Diagram	23
Figure 3.3.1 System Architecture Design	24
Figure 3.3.2 Backend Analysis Pipeline	27
Figure 3.4.1 System Flowchart	29
Figure 8.1.1 Project Timeline.....	72
Figure 8.2.1 User Test Case 1	73
Figure 8.2.2 User Test Case 2	74
Figure 8.2.3 User Test Case 3	75
Figure 8.2.4 User Test Case 4	76
Figure 8.2.5 User Test Case 5	77
Figure 8.2.6 User Test Case 6	78
Figure 8.2.7 User Test Case 7	79
Figure 8.2.8 User Test Case 8	80
Figure 8.2.9 User Test Case 9	81
Figure 8.2.10 User Test Case 10	82

List of Tables

Table 3.1.1 Phases for Agile Methodology	22
Table 3.7.1 UI Quality Condition Penalty	32
Table 3.7.2 UI Quality Classification	32
Table 5.1.1 Components Used for The Project	48
Table 5.2.1 Metrics Produced by The System	49
Table 5.3.1 Example Backend Analysis Output.....	50
Table 5.4.1 Issues Detected by The System.....	51
Table 5.5.1 Severity Distribution	53
Table 5.6.1 Distribution of UI/UX Issues by Category	54
Table 5.7.1 Findings and Recommendations by The System	55
Table 5.8.1 Average Time Used for Each Stage.....	57
Table 5.9.1 Functional Testing Results	58

1. Introduction

1.1. Introduction

The quality of User Interface (UI) and User experience (UX) has become a key factor in the success of modern digital products. UI/UX affects user satisfaction and engagement. However, the design process often faces significant challenges. Manually conducting comprehensive heuristic evaluations is time-consuming and can be inconsistent. This inconsistency arises from the subjective nature and varied expertise of human evaluators. As a result, this bottleneck delays critical feedback, leading to late discovery of usability issues, which is expensive and difficult to remediate during later development stages.

To address the need for accessible, this project proposes the development of an “AI Chatbot for UI Improvement”. This application will help UI/UX designers and developers to identify common user interface (UI) and user experience (UX) issues. The core of the system is a hybrid-logic chatbot that analyzes uploaded UI screenshots with 2 approaches:

- Rule-based Logic: Uses Python implementation to quickly identify basic and common UI issues based on predefined heuristics.
- Advanced AI Analysis: Uses Gemini 2.5 flash to analyze the visual data in the screenshot and provide more heuristic feedback and suggestions for improvement.

The application will be developed with Python-centric stack for prototyping and development. The backend will use FastAPI in Python to enable fast communication. Screenshot uploads are handled through a standard File upload interface, while the backend uses OpenCV to process the images for analysis. All data will be managed efficiently using MongoDB Atlas Database. Finally, the frontend will be developed using standard web technologies – HTML, CSS and JavaScript to deliver responsive chatbot interface.

The development of the “AI Chatbot for UI Improvement” supports several United Nations Sustainable Development Goals (SDGs) by promoting efficiency and access to quality tools.

One of the SGD is SDG 9: Industry, Innovation and Infrastructure. By automating the design review process, the tool directly supports innovation and helps build reliable infrastructure such as digital products. It improves their quality and makes them less likely to need expensive fixes. This leads to better productivity in the digital industry.

1.2. Problem Statement

Manually conducting a thorough heuristic evaluation is time-consuming and requires a high level of expertise. This makes it difficult for small teams and individual freelancers to conduct evaluations consistently. The process is highly dependent on evaluator's knowledge and experience, this leads to differences in findings due to subjective judgements (Göransson, 2004). As a result, the lack of standardization reduces the effectiveness of usability evaluations in real-world environments (Huang & Benyoucef, 2022).

Furthermore, the increasing complexity of modern applications presents another significant challenge. Mobile applications often include multiple features such as user authentication, in-app navigation and interactive elements across many screens. For example, a task like making a purchase may require users to move through several steps and pages. This makes it impractical to manually review every screen and interaction. Without a systematic approach, usability issues may be overlooked and making the design review process inefficient (Meniar et al., 2017). The difficulty of applying design guidelines consistently across complex interfaces aggravates this problem (Ballantyne et al., 2018).

In addition, usability issues are often identified too late in the development lifecycle due to the lack of immediate feedback during the early design phase. This delay increases the cost and effort required to fix the problem, as late changes in development are more expensive (Boehm & Basili, 2005). Over time, unresolved usability issues accumulate into “usability debt”, which can delay project delivery (Cai & Kazman, 2023). Limited availability of evaluators also contributes to this issue, as common usability problems may go unnoticed until later stages of development (Laokitpanich & Senivongse, 2022).

1.3. Objectives

The objectives are:

- To develop a tool that allows users to upload UI screenshots or describe UI screens and receive usability improvement suggestions
- To implement a rule-based engine that detects common UI/UX issues, enabling consistent evaluation across interfaces
- To integrate a lightweight AI model that provides enhanced usability feedback during the design phase

1.4. Research Questions

The goal of this project is to overcome the limitations of manual heuristic evaluation by providing fast and accessible analysis. Below are the five research questions that will guide the project's development:

1. How can a hybrid AI system balance objective consistency (through rule-based logic) with nuanced insights (through Gemini 1.5 flash) to generate actionable UI/UX improvement suggestions?
 - This question focuses on the core fusion methodology, investigating how to combine the output of Python rules with generative LLM to deliver the analysis.
2. What is the optimal technical pipeline required to process UI screenshot and transform it into a structured data format that maximizes the quality of the analysis output?
 - This question addresses the backend flow, focusing on file handling, image parsing using OpenCV within the FastAPI to ensure optimal performance.
3. How can the output of the rule-based logic be structured and integrated into the prompt to guide the Gemini 2.5 flash model to provide relevant feedback?
 - This question targets the prompt challenge, focusing on how the rule-based findings should be formatted to serve as reliable context for the LLM to enhance the precision of the analysis.
4. How does AI chatbots reduced “usability debt” bottleneck by providing actionable feedback compared to time-consuming manual evaluation?
 - This question relates to the project's value. It aims to accelerate the design review process and deliver timely feedback for the critical UI/UX issues.
5. What criteria must be implemented in the frontend design to ensure the AI chatbot provides responsive user experience when displaying complex analysis?

- This question focuses on the usability aspect. It looks into the best practices in designing the chat interface to present feedback and maintain user trust.

1.5. Solution Proposed

This project proposes the development of an intelligent “AI Chatbot for UI Improvement” designed to automate and enhance the heuristic evaluation process. The system aims to provide consistent and actionable UI/UX feedback by integrating rule-based logic with advanced AI analysis.

The proposed solution consists of a hybrid architecture with two main components. First, a rule-based engine implemented in Python analyzes uploaded UI screenshots using predefined usability heuristics. This component ensures consistency and detects common issues such as poor contrast, missing labels and layout misalignment. By standardizing these checks, the system reduces the subjectivity found in manual evaluation.

Second, the system incorporates an advanced AI model (Gemini 2.5 flash) to perform deeper analysis of the UI. It generates more nuanced suggestions such as enhancing accessibility and optimizing interaction design. The outputs from the rule-based engine are structured and included as part of the prompt to guide the AI model, ensuring more accurate and relevant feedback.

The overall workflow begins when the user uploads a UI Screenshot or provides a textual description. The backend is built with FastAPI, processes the input and uses OpenCV to extract visual features from the image. The rule-based engine then evaluates the screenshot and produces a structured report of detected issues. This report is passed to the AI model to generate enhanced recommendations. The final output is delivered through a chatbot interface.

This solution addresses the key challenges identified in the problem statement by reducing evaluation time, minimizing human bias, and providing immediate feedback. By automating the detection of usability issues and augmenting it with intelligent insights, the system helps designers identify and resolve problems earlier in the development process. As a result, it reduces

usability debt, improves design quality, and supports more efficient UI/UX workflows.

2. Literature Review

2.1. Literature Review and Options for Problem Solving

Improving User Interface (UI) design has become increasingly important as digital products compete to deliver better user experiences. A user-centric approach supported by data analysis and business intelligence helps designers to make informed decisions to improve usability and satisfaction (Gada & Chudasama, 2024). However, traditional UI evaluation methods are often manual, time consuming and require expert knowledge, creating a need for more efficient solution.

Artificial Intelligence (AI) has emerged a promising tool in enhancing User Experience (UX) design. Studies show that AI can assist designers during ideation and evaluation phases by generating alternative design solutions and supporting creative thinking (Khan et al., 2025). Additionally, AI-driven systems can personalize user experiences and provide adaptive feedback, although challenges such as integration complexity remain (Padmasiri *et al.*, 2023).

One critical aspect of UI Improvement is usability evaluation. Traditional approaches like heuristic evaluation based on Nielsen's principles rely heavily on expert judgement. Research highlights several limitations in automating these evaluations, including lack of formalization and insufficient tool support (Lecaros et al., 2021). Standardized questionnaires such as SAS, PSSUQ, QUIS and SUMI are also widely used, but they require user participation and are not suitable for real-time systems (Rotaru, O.A. *et al.*, 2020).

To address these limitations, recent research explores the use of Conversational User Interfaces (CUIs). CUIs allow users to interact with systems more naturally and reduce the complexity of traditional forms, making them effective for delivering technical support and feedback (Iniesto et., 2022). This suggests that chatbot-based systems can provide more accessible method for UI evaluation.

Furthermore, advancements in Generative AI and Large Language Models (LLMs) have enabled more intelligent analysis and reasoning capabilities. While LLMs can generate recommendations, studies show that they still face challenges in handling complex tasks and require better prompt design (Takaffoli et al., 2024). To improve reliability, the integration of rule-based logic with AI models, known as Neuro-Symbolic AI, has been proposed. This hybrid approach combines structured reasoning with the flexibility of machine learning, which improve decision-making and explainability (Xiong et al., 2024).

In addition to textual analysis, visual understanding plays a crucial role in UI evaluation. Recent developments in computer vision and multimodal AI enable systems to process images using visual prompting techniques, improving perception and reasoning in UI-related tasks (Wu et al., 2024). Frameworks such as Visual Task Prompt-based Image Processing (VIP) demonstrate how image-based inputs can be analyzed effectively, supporting the use of tools like OpenCV for extracting UI components (Chen et al., 2024).

Despite these advancements, existing solutions often focus on either automated evaluation, AI-based recommendations, or user interaction independently. There is a lack of integrated systems that combine conversational interfaces, AI reasoning, and visual analysis for comprehensive UI improvement.

Therefore, this project proposes a hybrid AI chatbot system that integrates rule-based heuristics with AI-driven recommendations and image processing capabilities. The chatbot will allow users to upload UI screenshots, analyze them using computer vision techniques, and receive interactive feedback through a conversational interface. This approach aims to address the limitations of existing methods by providing a more efficient, accessible, and intelligent solution for UI evaluation and improvement.

2.2. Preferred Option and Justifications

The proposed system adopts a hybrid architecture combining rule-based logic, AI models and image processing techniques to deliver effective UI improvement suggestions.

System Component:

- **Chatbot Logic**
 - The system will first apply rule-based heuristics derived from established UI principles such as Nielsen's heuristics and WCAG 2.2 to ensure consistent evaluations. For more advanced insights, the system will integrate Gemini 2.5 flash, which provides AI-generated recommendations and supports flexible reasonings. This hybrid approach aligns with the Neuro-Symbolic AI paradigm, improving both accuracy and interpretability

- **Database**
 - MongoDB Atlas will be used to store uploaded screenshots and evaluation results. Its NoSQL structure allows flexible handling of unstructured data such as chatbot logs and image metadata

- **Backend (FastAPI)**
 - FastAPI is selected for its high performance and support for asynchronous operations, making it suitable for handling real-time chatbot interactions and image processing tasks efficiently

- **Frontend (HTML, CSS and JavaScript)**
 - A lightweight web interface will be developed to ensure accessibility and ease of use. User can interact with the chatbot and upload UI screenshots through this interface

- **Screenshot Upload and Processing**
 - Users will upload UI screenshots, which will be processed using OpenCV. The system will extract visual elements and structure information to support analysis, aligning with recent advancements in computer vision for UI understanding.

3. Methodology

3.1. Methodology Used



Figure 3.1.1 Methodology Used: Agile

The development of the “AI Chatbot for UI Improvement” uses the agile development methodology. This approach is popular for its adaptability and flexibility, making it ideal for projects with complex iterations such as combining a rule-based system with the Google Gemini AI Model and evolving requirements (Beck et al., 2001). Agile methodology focuses on delivering the product in small steps rather than one big final result. This project will be broken down into short cycles. The core functions are built and tested in sequence to ensure a testable version of the project is available. The strength of the agile approach focuses on continuous feedback and adjustment. Regular meetings with the supervisor provide feedback, allowing for changes and updates problems arise. There are several phases in the agile methodology:

Table 3.1.1 Phases for Agile Methodology

Phases	Description
Planning	This phase involves defining the overall project title and the objectives, then a proposal is made. Problem statements and objectives are researched and features are prioritized to achieve the objectives
Design	This phase involves designing the core components for the project such as Backend using FastAPI, Database using MongoDB etc. UI sketches for the project are also designed
Development	This phase focuses on coding and implementing the techniques mentioned earlier in the design phase, where the designs are being transformed into usable and testable project
Testing	Testing phase is integrated after development during each cycle. This includes testing the core logic, integration and usability
Deployment	As the functional increments are completed, these increments can be prepared for the final online deployment to demonstrate real-world accessibility
Review and Documentation	This phase involves demonstrating the working features completed during the cycle. Feedback from the supervisor is collected and fed back into the next planning phase for maintenance. Final report Submission and project presentation is also involved

3.2. Use Case Diagram

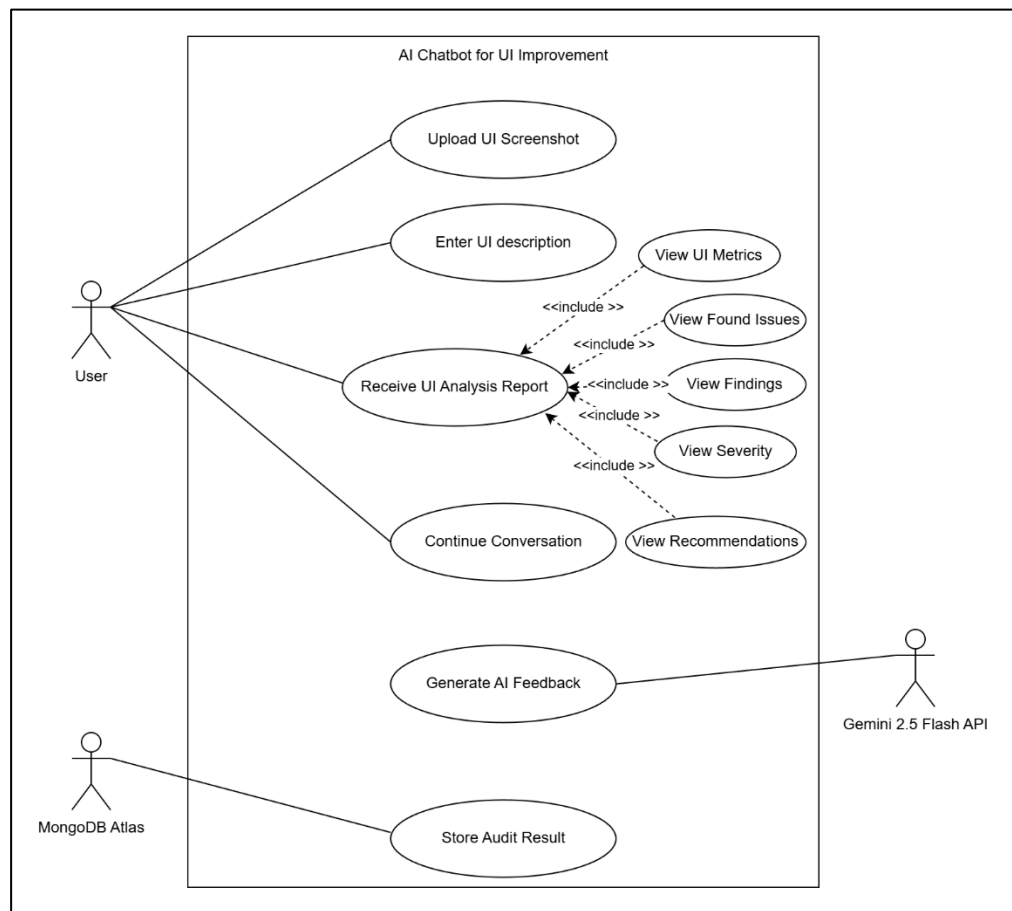


Figure 3.2.1 Use Case Diagram

The use case diagram illustrates the interactions between the user and the AI Chatbot for UI Improvement system. The primary actor is the User, who can upload a UI screenshot, enter a UI description, and continue the conversation through the chatbot interface. After receiving the input, the system generates a UI analysis report containing metrics, detected issues, usability findings, severity classifications, and improvement recommendations. The Google Gemini 2.5 Flash API acts as an external actor responsible for generating intelligent UI/UX feedback based on the provided screenshot and user query. MongoDB Atlas serves as the database actor, storing audit results for record-keeping and future analysis. This interaction enables automated and efficient UI evaluation.

3.3. System Architecture Design

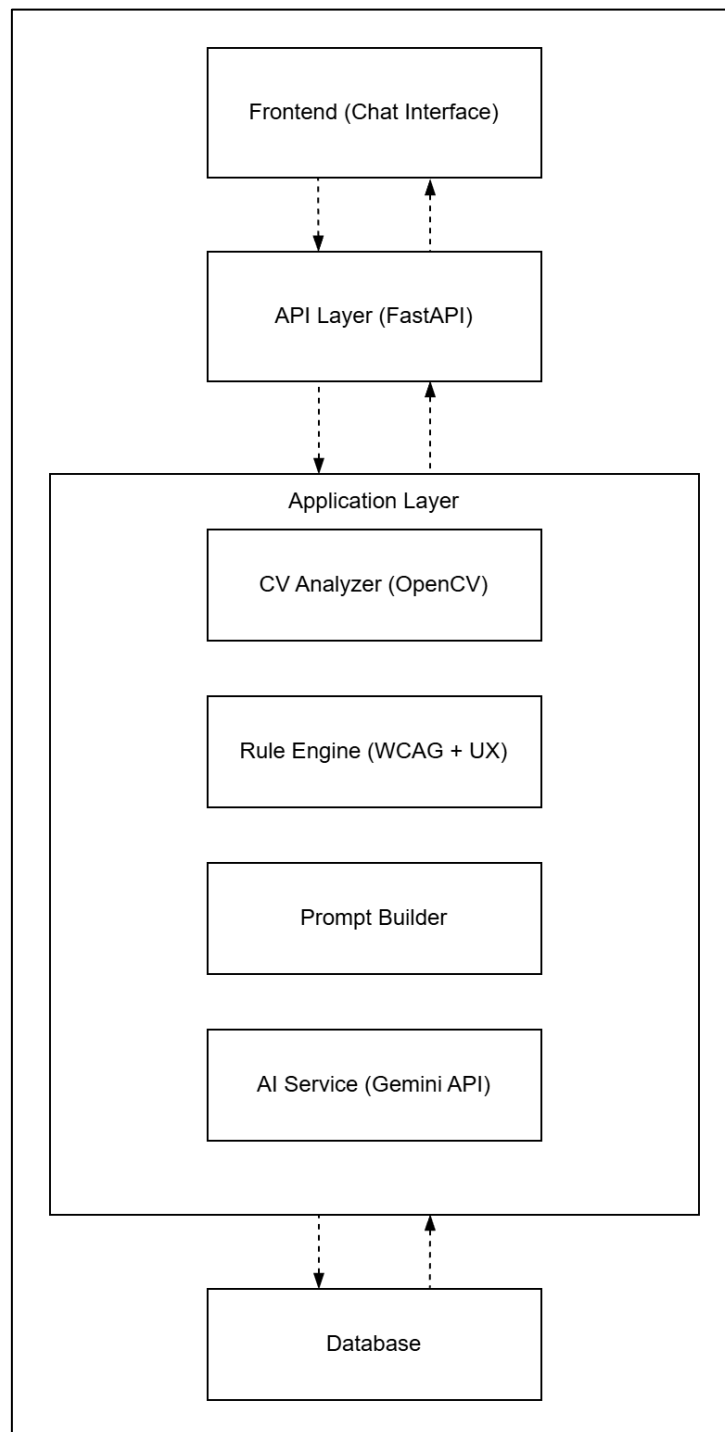


Figure 3.3.1 System Architecture Design

The system architecture is designed using a layered architecture approach. The system consists of four main layers, Frontend Layer, API Layer, Application Layer and Database Layer. Each layer performs specific tasks in processing user requests and generating suggestions.

The fronted layer acts as the user interface where user can interact with the system by uploading UI screenshots or enter textual descriptions of their UI designs. This layer is responsible for sending user requests to the backend services and displaying the generated suggestions.

The API layer is implemented using FastAPI and it acts as the bridge between the frontend and backend components. This layer handles incoming requests, validates uploaded files and user queries. The backend also maintains conversation continuity by storing and forwarding previous chat history to the AI model during subsequent requests.

The application layer contains the core functionalities of the system, including computer vision analysis, rule-based usability auditing, prompt engineering and AI response generation. The OpenCV-based CV Analyzer processes uploaded UI screenshots to extract visual usability metrics. The system performs image preprocessing operations to identify interface elements and usability issues.

The rule engine module is implemented using predefined rules based on WCAG 2.2 guidelines and Nielsen's heuristics. The engine evaluates interface components such as minimum contrast ratio, minimum touch target size etc. Severity levels are assigned based on the predefined weight configuration. This rule-based approach ensures the evaluation is consistent.

The rule engine also includes a UI quality classification module that converts extracted metrics into a numerical score. The system starts with a base score of 100 and applies weighted derived from WCAG accessibility thresholds and Nielsen's heuristics. The final score is then mapped into three categories: good, average and poor to help system to adjust the level of feedback generated by the AI model.

The prompt builder module constructs prompt by combining the extracted OpenCV analysis results, audit thresholds, heuristic definitions and user queries. These prompts are designed to guide the AI model to generate

relevant feedback. System instructions are applied to constrain the AI model so that the responses remain in specialized UI/UX auditing.

The AI service integration uses the Google Gemini 2.5 flash model to generate usability suggestions. The processed image and generated prompt are sent to the Gemini API through a chat session that supports conversational memory. The AI-generated results are then returned to the frontend and displayed to the user.

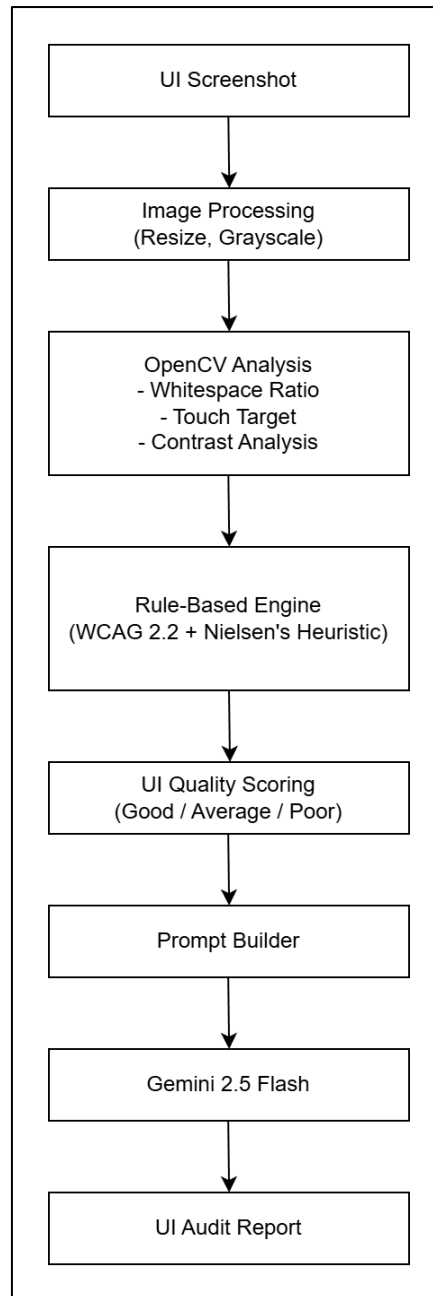


Figure 3.3.2 Backend Analysis Pipeline

Figure 3.3.2 illustrates the backend analysis pipeline implemented in the proposed system. The process begins when a UI screenshot is uploaded by the user. The screenshot is first processed by the OpenCV analysis module, where image preprocessing operations are performed to extract usability-related metrics such as whitespace ratio, touch target measurements, and contrast information.

The extracted metrics are then forwarded to the rule-based evaluation engine. This module applies predefined WCAG 2.2 accessibility thresholds and Nielsen's usability heuristics to identify potential usability violations. Based on the detected findings, a UI quality score and classification label (Good, Average, or Poor) are generated.

The resulting metrics, usability findings, and quality classification are subsequently passed to the prompt construction module. This module dynamically combines the extracted metrics, audit rules, and user query into a structured prompt. The generated prompt, together with the uploaded screenshot, is then submitted to the Gemini 2.5 Flash model.

Finally, the AI model generates usability recommendations guided by the quantitative metrics and rule-based findings produced by the preceding stages. This pipeline ensures that the AI analysis is grounded in objective usability measurements rather than relying solely on visual interpretation.

This pipeline forms the core computational contribution of the system, integrating deterministic computer vision analysis with rule-based reasoning and generative AI prompting.

3.4. System Flowchart

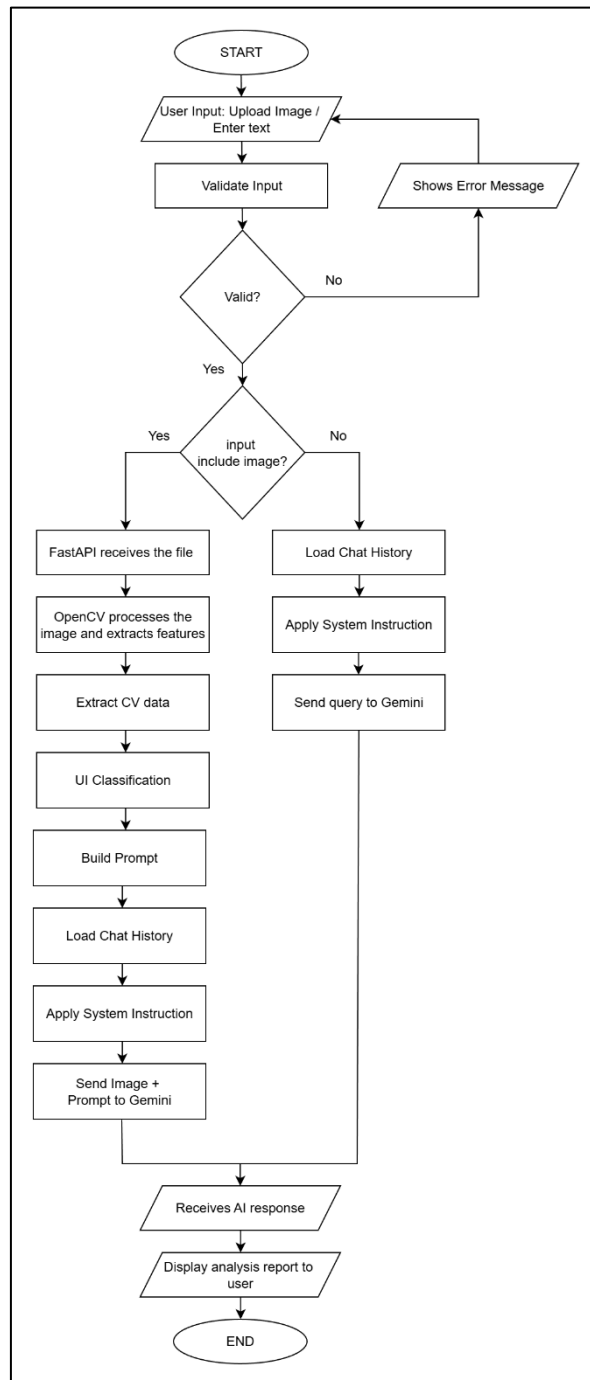


Figure 3.4.1 System Flowchart

The system flowchart illustrates the complete workflow of the system from user interaction to usability feedback generation. The process begins when the user interacts with the chat interface by either uploading a UI screenshot, entering textual description or combining both forms of input. Once the request is submitted, the system validates the input to ensure at least one valid image or query is provided before proceeding with the analysis process.

If the user submits only a textual query, the FastAPI backend initializes a chat session with the Google Gemini AI model and loads previous chat history to maintain conversational continuity. System instructions that containing constraints are applied to guide the AI response generation process. The query is sent to the Gemini Model directly, then the model generates recommendations and returns the response to frontend interface for display.

If the request contains an uploaded UI screenshot, the FastAPI backend reads the image data first and forwards it to the OpenCV computer vision module for preprocessing and analysis. The system identifies interface elements and extracts related metrics.

After image processing is completed, the backend executes the analysis pipeline described in Figure 3.3.2, including rule-based evaluation, UI quality classification, prompt construction, and AI-powered recommendation generation.

The classified results and extracted metrics are then passed to the prompt building module. The system constructs a prompt by combining WCAG thresholds, Nielsen's heuristics definitions, OpenCV analysis results and user queries. Then, the prompt and the image is sent to the Google Gemini AI model for advanced analysis.

The Gemini model generates a report containing identified issues, severity levels and improvement recommendations. Finally, the analysis report is displayed to the user through the chat interface.

3.5. Image Processing and Feature Extraction

- Uploaded screenshots are processed using OpenCV
- Image preprocessing techniques such as grayscale conversion are applied
- Whitespace ratio is calculated to estimate UI clutter and visual density
- Touch target sizes are measured using WCAG accessibility standards
- Processed images are being compressed before being sent to the AI model for analysis

3.6. Rule-Based Usability Evaluation

The system applies predefined usability rules stored in rules.py

Evaluation is based on WCAG 2.2 Accessibility Standards and Nielsen's Heuristics

Accessibility checks include minimum text contrast ratio and touch target size validation

Usability heuristic include:

- Visibility of system status
- Consistency and standards
- Error Prevention
- Recognition over recall
- Aesthetic and minimalist design

Severity levels are categorized into:

- Critical
- Major
- Minor

3.7. UI Quality Scoring Mechanism

Table 3.7.1 UI Quality Condition Penalty

Condition	Penalty
Whitespace ratio below 15%	-20
Whitespace ratio above 80%	-10
Small elements above 50	-25
Small elements above 20	-10
Contrast issues above 10	-25
Contrast issues above 5	-10

Table 3.7.2 UI Quality Classification

Score Range	Classification
80-100	GOOD
50-79	AVERAGE
Below 50	POOR

The UI quality score begins from a base score of 100. Penalties are deducted based on detected usability and accessibility issues, including low whitespace ratio, excessive whitespace, small interactive elements, and contrast failures. The final score is then converted into a UI quality classification of GOOD, AVERAGE, or POOR. This classification is used to guide the Gemini prompt, allowing the AI model to adjust the severity and tone of its feedback.

3.8. AI-Powered Feedback Generation

- The backend integrates Google Gemini 2.5 Flash for intelligent UI Auditing
- The AI model receives:
 - Processed Screenshot image
 - Computer vision metrics
 - UX audit instructions
 - Optional user queries
- AI-generated feedback includes:
 - UX findings
 - Accessibility Violations
 - Severity classifications
 - Suggested Improvement

3.9. Prompt Engineering Strategy

- A structured system prompt is used to define the AI's role as a "Senior UI/UX Auditor"
- Prompt Engineering ensures consistent response formatting and reduced irrelevant responses
- Prompts are generated using:
 - OpenCV analysis metrics
 - Rule-based audit threshold
 - User questions
- The prompt contains:
 - Accessibility Standards
 - Heuristic definitions
 - Severity guidelines
 - Output formatting constraints

3.10. Database

- The system integrates a NoSQL database using MongoDB
- Database connectivity is implemented through the Motor asynchronous MongoDB Driver
- Environment variables are managed using python-dotenv
- Audit records are stored in:
 - Database: ui_audit_db
 - Collection: audits
- Each audit document contains:
 - Timestamp of analysis
 - Uploaded filename
 - User query
 - OpenCV metrics
 - AI-generated audit report

4. Implementation

4.1. System Architecture Overview

The “AI Chatbot for UI Improvement” is built using a modular client-server architecture consisting of frontend, backend, computer vision processing, rule-based auditing, AI integration and cloud database components. The implementation focuses on providing real-time UI/UX auditing using a conversational chatbot interface.

The frontend interface is built with HTML, CSS and JavaScript that allows users to upload UI screenshots and submit usability-related questions. User requests are sent asynchronously to the FastAPI backend using HTTP POST requests and multipart form data.

The backend acts as the central controller of the system. It handles request validation, manages image processing operations and coordinates communication with the Google Gemini 2.5 Flash AI model. The detailed backend processing sequence is illustrated in Figure 3.3.2, where OpenCV analysis results are evaluated through a rule-based engine before being incorporated into AI prompt generation. Uploaded screenshots are processed using OpenCV computer vision techniques in the `cv_analysis.py` module.

A rule-based auditing system is implemented in `rules.py` provides predefined usability and accessibility standards based on WCAG 2.2 and Nielsen’s heuristics. The extracted OpenCV metrics and rule definitions are dynamically integrated into prompts before being sent to the Gemini AI model.

Generated audit reports are returned to the frontend chatbot interface and simultaneously stored in MongoDB Atlas using asynchronous database operations through the Motor MongoDB Driver.

4.2. Frontend Implementation

4.2.1. Chat Interface Design

The user interface is implemented in index.html. It consists of:

- Chat display container
- User text input field
- Screenshot Upload button
- Image Preview modal

Users can either send text-only queries for suggestions or send screenshots for auditing.

4.2.2. Asynchronous Communication

JavaScript Fetch API is used to establish asynchronous communication between the frontend and FastAPI Backend. User requests are submitted using multipart form data through HTTP POST requests.

The `processUpload()` function is responsible for:

- Packaging uploaded files
- Sending user queries
- Forwarding chat history
- Receiving AI-generated responses

This asynchronous implementation allows the interface to remain responsive while backend processing.

4.2.3. Conversational Interface Features

Some chatbot interaction features are implemented such as:

- Dynamic typing animation using `typeEffect()`
- Automate textarea resizing using `autoGrow()`
- Image preview and modal enlargement
- Markdown rendering using the `Marked.js` library
- Persistent chat history tracking

The system stores recent conversation history locally in JavaScript arrays before forwarding it to the backend to maintain conversational continuity.

4.3. FastAPI Backend Implementation

The backend system is implemented using FastAPI due to its high performance, asynchronous processing support and efficient API development capabilities.

4.3.1. API Initialization

The main backend implementation is located in `FastAPI_Backend.py`. The application initializes:

- FastAPI application instance
- Cross-Origin Resource Sharing (CORS) middleware
- Google Gemini client
- Environment variable configuration
- Logging Services

CORS middleware allows communication between the frontend interface and backend server during development.

4.3.2. API Endpoint Implementation

The primary API endpoint implemented is: /analyze/ui-screenshot

This endpoint accepts:

- Uploaded screenshot images
- User textual queries
- Previous conversation history

The endpoint supports:

- Text-only conversations
- Image-based UI audits
- Combined image and text analysis

4.3.3. Request Validation

Input validation is implemented to ensure that users provide at least:

- A valid screenshot image
- A textual query

Invalid requests trigger HTTP 400 exceptions using FastAPI's exception handling system

4.3.4. Asynchronous Processing

The backend uses asynchronous functions through Python's async and await syntax to improve performance during:

- File uploads
- Image processing
- Database storage
- AI model communication

This asynchronous implementation improves responsiveness when handling large screenshots and external API requests.

4.3.5. Error Handling and Retry Mechanism

The backend includes an error handling and retry mechanism for Gemini API communication. If the Gemini service temporarily returns a 503 or unavailable error, the system does not immediately fail the request. Instead, the backend retries the request up to four times using exponential backoff. This improves system reliability when external AI services experience temporary outages.

If the error is not related to temporary service unavailability, the backend raises the exception immediately. This prevents unnecessary retries for invalid requests or configuration errors.

4.4. OpenCV Computer Vision Implementation

The OpenCV analysis module is implemented in `cv_analysis.py` to perform automated UI screenshot auditing.

4.4.1. Image Preprocessing

Uploaded screenshots are first converted into NumPy arrays and decoded using OpenCV image decoding functions.

Several preprocessing operations are applied:

- Image resizing
- Grayscale conversion
- Adaptive thresholding
- Edge detection using the Canny algorithm (Canny, 1986)

These preprocessing steps improve the accuracy of UI element detection.

4.4.2. Whitespace Analysis

The `calculate_whitespace_ratio()` function estimates interface clutter by analyzing empty pixel regions.

The implementation uses:

- Edge detection
- Morphological dilation
- Pixel density calculations

Low whitespace ratios indicate visually cluttered interfaces and may violate the “Aesthetic and Minimalist Design” heuristic (Nielsen, 1994).

4.4.3. Touch Target Analysis

Contour detection techniques are used to identify interactive UI components.

The system evaluates bounding box dimensions against WCAG 2.2 touch target requirements:

- AA Standard: 24 x 24 pixels
- AAA Standard: 44 x 44 pixels

Small interactive elements are highlighted using yellow bounding box for debugging visualizations (W3C, 2024).

4.4.4. Contrast Ratio Analysis

The system calculates relative luminance using WCAG contrast formulas implemented in:

- `get_luminance()`
- `calculate_contrast()`

Average region contrast ratios are computed and compared against accessibility thresholds. Elements that fail minimum contrast requirements are highlighted using red bounding boxes (W3C, 2024).

4.4.5. Visual Debugging Output

After the analysis, the system generates annotated screenshot that contains yellow indicators for small touch target and red indicators for contrast failure. These annotated images are compressed and sent to the Gemini AI model to improve contextual understanding.

4.5. Rule-Based Expert System

The rule-based auditing engine is implemented in `rules.py` to provide standardized usability evaluation.

4.5.1. WCAG Accessibility Rules

The system stores predefined accessibility thresholds such as:

- Minimum text contrast ratios
- UI component contrast requirements
- Minimum touch target dimensions

These thresholds are referenced during OpenCV analysis and AI prompt construction.

4.5.2. Nielsen's Heuristics Integration

The system integrates Nielsen's heuristics to act as structured Python dictionaries for dynamic prompt generation.

4.5.3. UI Quality Classification

A rule-based scoring mechanism is implemented in `FastAPI_Backend.py` to evaluate overall UI quality.

The scoring process begins with a base score of 100. Penalty values are applied according to detected usability issues, including whitespace ratio violations, excessive numbers of small interactive elements, and contrast failures.

The implemented scoring rules are:

- Whitespace ratio below 15%: -20
- Whitespace ratio above 80%: -10
- Small elements above 50: -25
- Small elements above 20: -10
- Contrast issues above 10: -25
- Contrast issues above 5: -10

The final score is classified into:

- GOOD (80–100)
- AVERAGE (50–79)
- POOR (Below 50)

This classification is subsequently used during prompt construction to control the severity and tone of AI-generated recommendations.

4.5.4. Severity Classification

The system categorizes usability findings into Critical, Major and Minor. Each severity level is associated with predefined weighting values to support future scoring system expansion.

4.5.5. Rule-to-AI Prompt Bridge

The `get_rules_string()` function converts Python rule definitions into structured textual instructions. This implementation acts a bridge between deterministic rule-based auditing and generative AI reasoning.

4.6. Prompt Engineering and Gemini AI Integration

The system integrates Google Gemini 2.5 Flash to generate advanced UI/UX recommendations.

4.6.1. System Instruction Design

A structured system instruction is implemented to constrain the AI model behaviour. The system instruction defines:

- AI personality
- Scope restrictions
- Supported heuristics
- Accessibility thresholds
- Output formatting rules

This prevents unrelated responses and improves audit consistency.

4.6.2. Dynamic Prompt Construction

The `build_audit_prompt()` function dynamically generates prompts using OpenCV metrics, rule-based thresholds, user queries and heuristic definitions. The generated prompt contains technical audit rules, image analysis facts, user context and structured output constraints.

4.6.3. Structured Response Formatting

The AI model is instructed to generate finding title, severity classification and practical fix recommendations. The response format is limited to three structured findings to improve readability.

4.6.4. Conversational Memory

Previous chat messages are stored and sent to Gemini chat session using:

```
chat = client.chats.create(  
    model=model_id,  
    config=config,  
    history=past_messages  
)
```

This allows the chatbot to maintain contextual continuity during multi-turn interactions.

4.7. MongoDB Database Implementation

MongoDB Atlas is implemented as the cloud-based NoSQL database solution for storing audit records.

4.7.1. Database Connectivity

Database communication is implemented in database.py using:

- Motor asynchronous MongoDB Driver
- TLS certificate validation
- Environment variable configuration

4.7.2. Audit Record Storage

The system stores audit results in:

- Database: ui_audit_db
- Collection: audits

Each document contains timestamp, uploaded filename, user query, OpenCV metrics and Gemini-generated report.

4.7.3. Asynchronous Database Operations

The `save_audit()` function stores records asynchronously to avoid blocking API operations during database communication.

4.8. Overall System Workflow

The workflow begins when the user uploads a screenshot or submits a UI-related query through the chatbot interface.

The backend processing follows the Backend Analysis Pipeline shown in Figure 3.3.2, where OpenCV metrics are evaluated using WCAG and Nielsen-based rules before being integrated into the AI prompt construction process.

5. Findings and Results

5.1. Experimental Setup and Parameters

The system is evaluated using multiple UI screenshots collected from web and mobile interfaces. The testing process focuses on analysing the chatbot's ability to detect UI/UX issues, generate usability recommendations and maintain responsive performance.

The development and testing environment specifications are shown in Table 5.1.1.

Table 5.1.1 Components Used for The Project

Component	Specification
Backend Framework	FastAPI
Programming Language	Python
AI Model	Gemini 2.5 flash
Database	MongoDB Atlas
Computer Vision Library	OpenCV
Frontend	HTML, CSS and JavaScript
Browser used	Google Chrome
Operating System	Windows 11
Number of UI screenshots tested	10

The testing process involved:

- Uploading screenshots into the chatbot interface
- Running OpenCV-based usability analysis
- Generating AI-powered UI/UX recommendations
- Measuring response performance
- Verifying database storage functionality

5.2. Technical Metrics Analysis

The system extracts quantitative UI metrics using OpenCV. These metrics are used as indicators of UI complexity and accessibility.

From testing, the system produced the following types of metrics:

- Whitespace ratio (%)
- Numbers of small interactive elements

Table 5.2.1 Metrics Produced by The System

Screenshot	Whitespace (%)	Small Elements
Screenshot 1	49.62	26
Screenshot 2	69.25	39
Screenshot 3	70.41	10
Screenshot 4	71.32	40
Screenshot 5	69.05	47
Screenshot 6	82.8	19
Screenshot 7	87.83	176
Screenshot 8	71.4	121
Screenshot 9	11.94	114
Screenshot 10	76.63	133

The results indicate that screenshots with lower whitespace percentages generally exhibit denser visual layouts. Screenshot 9 demonstrates a combination of extremely low whitespace availability and a high number of small interactive elements, suggesting a high risk of usability and accessibility violations. Screenshot 10, although not visually dense, contains a large number of small interactive elements that may still affect usability and accessibility.

5.3. Backend Analysis Output

Table 5.3.1 Example Backend Analysis Output

Metric	Value
Whitespace Ratio	70.57%
Is cluttered	False
Small Interactive Elements Found	129
Contrast Issues Found	113
Average Contrast Ratio	2.2
UI Quality Score	50
UI Quality Classification	AVERAGE

The backend analysis pipeline generates structured usability metrics before AI-powered recommendation generation. Table 5.3.1 presents an example of the output produced by the OpenCV analysis module and rule-based evaluation engine.

For the tested screenshot, the system detected a whitespace ratio of 70.57%, indicating that the interface was not considered visually cluttered according to the predefined threshold. However, the analysis identified 129 small interactive elements and 113 contrast-related issues. The calculated average contrast ratio was 2.2, which falls below the WCAG 2.2 minimum requirement of 4.5:1 for normal text.

These metrics were subsequently processed using the UI quality scoring mechanism described in Section 3.7. Based on the implemented penalty rules, the screenshot obtained a final score of 50 and was classified as AVERAGE. The generated metrics and classification were then incorporated into the prompt construction process and supplied to the Gemini 2.5 Flash model to guide recommendation generation.

This result demonstrates that the system performs deterministic backend analysis before AI inference. Rather than relying solely on generative AI interpretation, the proposed architecture first extracts measurable usability indicators and evaluates them using predefined accessibility and usability rules. The AI model therefore acts as a reasoning and recommendation layer built upon objective backend measurements.

5.4. Rule-Based Usability Detection Results

The system applied WCAG 2.2 guidelines and Nielsen's heuristics to detect UI issues.

Table 5.4.1 Issues Detected by The System

Screenshot	Detected Issues	Severity
Screenshot 1	Insufficient Colour Contrast	Critical
Screenshot 1	Small Interactive Elements	Major
Screenshot 1	Visual Clutter and Hierarchy	Major
Screenshot 2	Small Interactive Elements	Critical
Screenshot 2	Insufficient Colour contrast	Major
Screenshot 2	Overwhelming promotional content	Major
Screenshot 3	Insufficient contrast for inactive tab state	Major
Screenshot 3	Unclear form field labels	Major
Screenshot 3	Error message clarity and terminology	Minor
Screenshot 4	Inadequate touch target sizes	Major
Screenshot 4	Insufficient color contrast	Major
Screenshot 4	Ambiguous interactive visual cues	Minor
Screenshot 5	Insufficient text contrast in sidebar	Minor
Screenshot 5	Potentially small touch targets	Minor
Screenshot 6	Undersized interactive elements	Major
Screenshot 6	Limited visual distinction for inactive states	Minor
Screenshot 6	Limited guidance for formatted input fields	Minor
Screenshot 7	Limited visual clarity of navigation bar focus state	Minor
Screenshot 7	Potential contrast reduction in focused navigation state	Major
Screenshot 7	Possible inconsistency in visual feedback for active elements	Minor
Screenshot 8	Insufficient contrast for key information	Major
Screenshot 8	Undersized interactive elements	Major
Screenshot 8	Inconsistent visual hierarchy and map legibility	Minor
Screenshot 9	Overwhelming visual clutter and lack of hierarchy	Critical
Screenshot 9	Undersized interactive elements	Critical
Screenshot 9	Pervasive low contrast text and UI components	Major

Screenshot 10	Critical contrast issues across UI elements	Critical
Screenshot 10	Insufficient target sizes for interactive elements	Major
Screenshot 10	Visual clutter and weak information hierarchy	Major

The rule-based engine successfully identified common usability violations such as insufficient colour contrast and visual clutter.

5.5. Severity Distribution Analysis

The detected issues are categorized into three severity level:

- Critical
- Major
- Minor

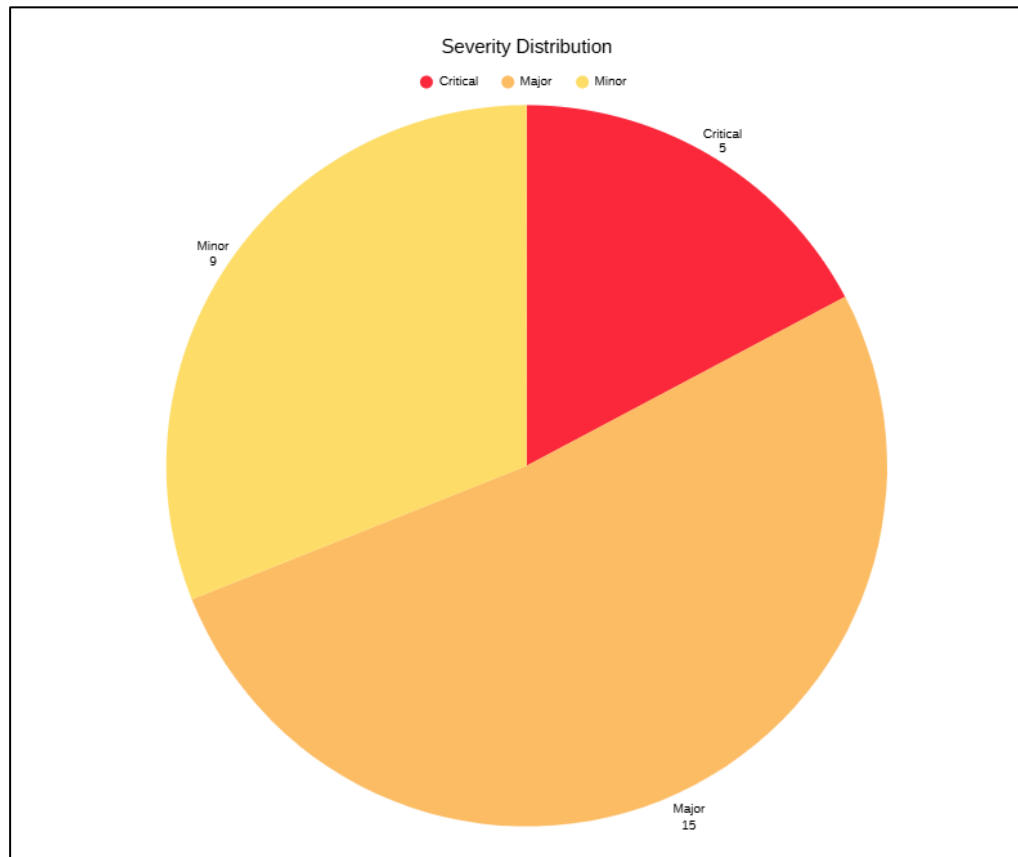


Table 5.5.1 Severity Distribution

Most detected issues fall under the Major Category, showing that frequent usability inconsistencies across tested UI designs. Critical Issues are primarily related to colour contrast and small interactive elements.

5.6. Usability Issue Category Analysis

The detected findings are also grouped according to usability issue categories.

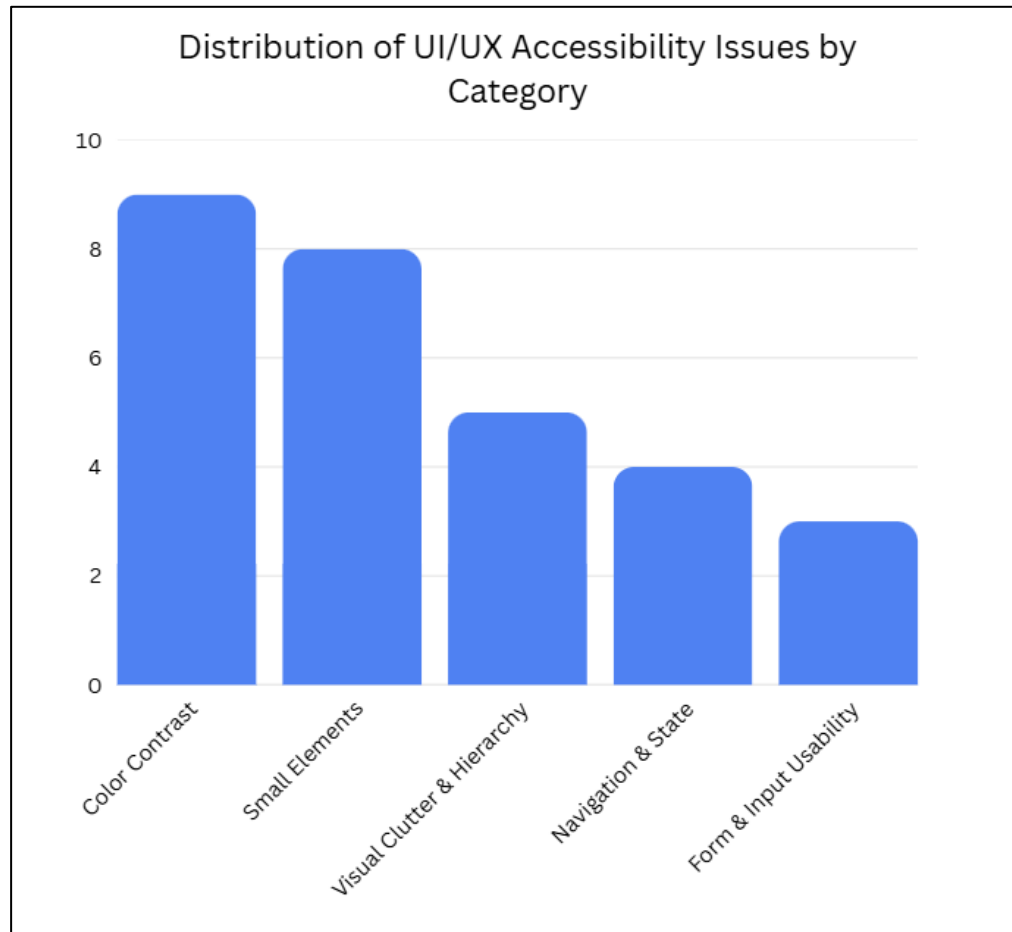


Table 5.6.1 Distribution of UI/UX Issues by Category

Based on the results, contrast-related problems are the most frequently detected issues, means that many tested interfaces fail to meet minimum WCAG contrast requirements. Touch target issues are also commonly detected.

5.7. AI-generated Feedback Evaluation

The Gemini 2.5 Flash is evaluated based on its ability to generate relevant UI/UX recommendations using the structured prompts.

Table 5.7.1 Findings and Recommendations by The System

Screenshot	AI Findings	Recommendations
1	Low contrast text	Improve WCAG contrast ratio
1	Small touch targets	Increase clickable area size
1	Dense layout hierarchy	Add whitespace and improve grouping
2	Small touch target sizes	Increase interactive elements to minimum 44×44px
2	Low contrast text components	Improve contrast ratios to WCAG AA standards
2	Excessive promotional visual noise	Simplify banners and improve visual hierarchy
3	Low visibility inactive tab text	Improve contrast and inactive tab styling
3	Missing persistent form labels	Add visible labels for all input fields
3	Generic error message wording	Use clearer and more professional validation feedback
4	Small clickable dashboard elements	Increase touch target size and spacing
4	Low contrast text and status indicators	Improve WCAG-compliant color contrast
4	Unclear interactive labels and tags	Use consistent visual indicators for clickable elements
5	Low contrast sidebar text	Improve sidebar text contrast ratio
5	Small clickable icons and controls	Increase interactive touch target areas
6	Small form interaction controls	Increase touch target size
6	Inactive selections appear visually subtle	Improve selected/unselected state differentiation
6	Date and phone input formats may be unclear	Add clearer inline formatting guidance
7	Navigation focus relies mainly on background darkening	Improve active state visibility using combined visual cues (color, highlight, or border)
7	Foreground elements may lose emphasis in focused state	Ensure text and icons remain clearly readable in all states

7	Interaction feedback could be more consistent	Standardize focus indicators across navigation components
8	Low contrast text for order and status elements	Improve WCAG AA contrast for key labels and inactive states
8	High number of small interactive elements (121 detected)	Increase touch target size to minimum 24×24px (ideal 44×44px)
8	Dense map and weak information hierarchy	Improve visual hierarchy and reduce map label clutter
9	Extremely dense layout with minimal whitespace	Increase spacing and reduce number of visible items per view
9	Large number of small clickable elements (114 detected)	Enlarge touch targets to meet WCAG minimum accessibility standards
9	Widespread low contrast across text and UI elements	Apply WCAG AA-compliant color palette for text and UI components
10	Widespread low contrast between text and background	Apply WCAG AA-compliant color palette for all text and UI components
10	Large number of small interactive elements (133 detected)	Increase touch target size to minimum 24×24px (ideal 44×44px)
10	Repetitive icons create visual noise and reduce clarity	Simplify icon usage and improve content grouping and hierarchy

The AI model is able to provide feedback based on the context. By combining rule-based inputs and visual metrics, the system generated usability recommendations that align with UX design principles.

5.8. System Performance Evaluation

The system performance is measured based on processing time for each stage.

Table 5.8.1 Average Time Used for Each Stage

Process	Average Time (seconds)
Image Upload	0.198
OpenCV Processing	0.03
AI Response Generation	14.757
Database Saving	0.021
Total Processing Time	15.663

The performance evaluation results show that the system is able to process UI screenshots within acceptable response time. The average total processing time of 15.663 seconds demonstrates that the system can provide usability feedback within a short period compared with traditional manual heuristic evaluation.

Among all the stages, AI response generation has the highest average processing time at 14.757 seconds. This shows that the latency is from the Gemini 2.5 Flash model during reasoning and recommendation generation. The AI model is responsible for interpreting the screenshot, analysing the extracted metrics and generate usability recommendations based on WCAG 2.2 and Nielsen's heuristics. Higher processing time is expected due to the complexity of large language model inference.

5.9. Functional Testing Results

Table 5.9.1 Functional Testing Results

Feature	Expected Result	Actual Result	Status
Screenshot Upload	Image uploaded	Success	Pass
UI Analysis	Metrics generated	Success	Pass
AI Feedback	UX suggestions shown	Success	Pass
Database Storage	Data saved	Success	Pass
Chat History	Maintain	Success	Pass

5.10. Discussion of Findings

The findings from evaluation demonstrates the proposed AI-powered UI auditing chatbot is able to detect common usability and accessibility issues. The system demonstrates a structured computational pipeline where UI screenshots are transformed into quantitative metrics using computer vision techniques. These metrics are then processed through a rule-based decision engine before being used to condition the generative AI model.”

A key strength of the system is its hybrid approach. The rule-based engine ensures consistency by applying deterministic WCAG 2.2 standards and Nielsen’s heuristics, particularly in detecting issues such as insufficient colour contrast, small touch targets, and unclear visual hierarchy. This reduces subjectivity compared to traditional manual heuristic evaluation, where different evaluators may produce inconsistent results.

The AI component further enhances the system by translating structured metrics into human-readable recommendations. The Gemini 2.5 Flash model is able to provide contextual suggestions that go beyond rule violations, such as improving layout grouping, enhancing visual hierarchy, and refining interaction feedback. This demonstrates the advantage of combining symbolic rules with generative AI for UI analysis.

However, some limitations were observed during evaluation. In certain cases, the system produced false-positive results, particularly in colour contrast and touch target detection, where acceptable designs were flagged as problematic. This indicates that while the rule-based module provides strong baseline detection, it is still dependent on image processing accuracy and threshold tuning. Additionally, the AI-generated outputs may vary slightly between runs, which affects consistency.

Overall, the results show that the system is effective in automating early-stage UI/UX evaluation, but human validation is still required for final design decisions. The system should therefore be viewed as a supportive tool rather than a complete replacement for expert usability evaluation.

6. Limitations, Future Enhancement, Risk Register, Discussion and Conclusion

6.1. System Limitations

Despite successfully achieving the project objectives, the project still has several limitations.

6.1.1. Technical Limitations

- **AI Response Latency:** The Google Gemini 2.5 Flash model requires a longer processing time compared to the OpenCV analysis module. Most of the overall response delay is from AI reasoning and recommendation generation.
- **Limited UI Element Detection Accuracy:** The OpenCV-based detection process relies on contour analysis and image preprocessing techniques. Complex interfaces may reduce detection accuracy for touch target size and contrast regions.
- **False Positive Findings:** Some AI-generated usability findings require manual verification. In some cases, the system may incorrectly classify acceptable UI designs as problematic due to limitations in visual interpretation.

6.1.2. AI and Prompt Engineering Limitations

- **Dependency on Prompt Quality:** The quality of AI-generated recommendations relies heavily on the structure and clarity of prompts generated by the backend system.
- **Model Interpretation Variability:** Although system instructions are applied, the Gemini model may produce slightly inconsistent responses.
- **Limited Explainability:** Some AI-generated recommendations may lack detailed explanations even the rule-based engine provides deterministic findings.
- **Lack of UI Screenshot Validation:** The system does not verify if uploaded images are UI screenshots, allowing non-UI images like selfies or photos to be processed, which may produce irrelevant or inaccurate UI/UX audit results.

6.1.3. Evaluation Limitations

- **Limited Number of Test Screenshots:** The evaluation was conducted using limited number of screenshots across selected UI designs. Additional testing could further validate system robustness.
- **Manual Verification Requirement:** Human verification is still needed to confirm the accuracy of certain AI-generated findings and severity classifications.
- **Static Screenshot Evaluation:** The system only evaluates uploaded screenshots and cannot analyse live applications or interactive transitions.

6.2. Future Enhancement

Several enhancements can be implemented to improve the system functionality in future development.

6.2.1. AI and Analysis Improvements

- **Advanced UI Component Recognition:** Future implementation can integrate deep learning-based object detection models to improve UI elements recognition accuracy.
- **Multimodal AI Integration:** More advanced multimodal models can be integrated to improve contextual understanding between visual components and textual descriptions.
- **Improved Prompt Engineering:** Adaptive prompt techniques can be implemented to adjust prompt complexity dynamically.
- **UI Screenshot Validation Module:** preprocessing step can be implemented to detect whether an uploaded image is a valid user interface screenshot. This would prevent non-UI images from entering the analysis pipeline and improve accuracy of audit results

6.2.2. System Enhancements

- **User Authentication System:** Future implementation may include user account management.
- **Dashboard Analytics:** A dashboard can be added to visualize usability trends and audit statistics.
- **Performance Optimization:** Response Latency can be reduced by implementing caching mechanisms and optimizing AI request pipelines.

6.2.3. Feature Expansions

- **Multi-language Support:** The chatbot interface and AI analysis can be expanded to support multiple languages.
- **Interactive Fix Suggestions:** The chatbot can provide visual redesign suggestions to show usability improvements in a clearer way.
- **Accessibility Compliance Reports:** Future versions can generate downloadable compliance reports in PDF format for auditing purposes.

6.3. Risk Register

6.3.1. Technical Risks

- **API Dependency Risk:** The system relies heavily on external API services. Service outages may affect system availability.
- **Performance Bottlenecks:** Large image uploads and AI processing requests may increase server workload.
- **Database Connectivity Failures:** Network interruptions or cloud database service failures may prevent audit records from being stored.

6.3.2. Security and Privacy Risk

- Sensitive Screenshot Uploads: Uploaded screenshot may contain user information, creating privacy and security concerns.
- Unauthorized Access: Without proper authentication, audit records can potentially be exposed to unauthorized users.
- Data Transmission Risks: Improperly secured API communication channels may expose uploaded images and chatbot conversations to interception.

6.3.3. Ethical Concerns

- AI Bias and Inconsistent Recommendations: The AI model may generate biased or inconsistent usability recommendations based on the prompt interpretation.
- Overdependence on Automation: Designers may rely on automated auditing and overlook the importance of human-centred usability evaluation.
- Explainability Challenges: Certain AI-generated suggestions may be difficult to justify due to the black-box nature of the system.

6.4. Discussion of Results

6.4.1. Hybrid System Effectiveness

The evaluation results show that combining rule-based usability auditing with AI-generated reasoning improves the effectiveness of UI evaluation. The OpenCV analysis module provides objective metrics and the Gemini 2.5 Flash generates recommendations.

6.4.2. Rule-based consistency

The rule-based engine successfully detected common usability violations such as insufficient colour contrast and undersized touch targets. The evaluation process helps reduce subjectivity that are common in manual heuristic evaluation.

6.4.3. AI-Powered Recommendation Quality

The Gemini AI model generated recommendations that are aligned with WCAG 2.2 guidelines and Nielsen's heuristics. The structured prompt improved the relevance of generated suggestions.

6.4.4. System Performance

The system demonstrated acceptable performance with an average total processing time of about 15 seconds per screenshot analysis. OpenCV processing and database operation are highly efficient. AI response generation contributed the majority of processing time.

6.5. Conclusion

This project successfully designed, developed, and evaluated an AI-powered chatbot for UI improvement that integrates computer vision, rule-based usability analysis, and generative AI reasoning. The system was built using a FastAPI backend, OpenCV for image processing, a rule-based engine based on WCAG 2.2 accessibility standards and Nielsen's heuristics, and the Gemini 2.5 Flash model for generating usability recommendations.

The proposed hybrid architecture effectively addresses the limitations of traditional manual heuristic evaluation by providing faster and more consistent feedback on UI designs. Through OpenCV-based analysis, the system extracts measurable usability indicators such as whitespace ratio, touch target sizes, and contrast levels. These metrics are then evaluated using predefined rules to ensure objective consistency before being passed to the AI model for enhanced interpretation and recommendation generation.

The results show that the system is capable of identifying key usability issues and producing relevant improvement suggestions that align with established UI/UX design principles. In addition, the average processing time of approximately 15 seconds demonstrates that the system is capable of delivering timely feedback suitable for early-stage design evaluation.

Despite its effectiveness, the system still has limitations such as occasional false-positive detections, dependency on prompt quality, and the inherent variability of AI-generated responses. Therefore, the system is best used as a supporting tool to assist designers rather than replace expert evaluation.

In conclusion, the project demonstrates that combining rule-based systems with generative AI and computer vision is a practical and effective approach for automating UI/UX evaluation. It improves design feedback efficiency, reduces usability evaluation effort, and supports the development of more accessible and user-friendly digital interfaces.

7. References

Luther, L., Tiberius, V., & Brem, A. (2020). User Experience (UX) in Business, Management, and Psychology: A bibliometric mapping of the current state of research. *Multimodal Technologies and Interaction*, 4(2), 18. <https://doi.org/10.3390/mti4020018>

Virvou, M. (2023). Artificial Intelligence and User Experience in reciprocity: Contributions and state of the art. *Intelligent Decision Technologies*, 17(1), 73–125. <https://doi.org/10.3233/idt-230092>

Stige, Å., Zamani, E. D., Mikalef, P., & Zhu, Y. (2023). Artificial intelligence (AI) for user experience (UX) design: a systematic literature review and future research agenda. *Information Technology and People*, 37(6), 2324–2352. <https://doi.org/10.1108/itp-07-2022-0519>

Khan, A., Shokrizadeh, A., & Cheng, J. (2025). Beyond Automation: How UI/UX Designers Perceive AI as a Creative Partner in the Divergent Thinking Stages. *ArXiv (Cornell University)*. <https://doi.org/10.1145/3706598.3713500>

Bengt Göransson. (2004, June). *User-Centred Systems Design - Designing Usable Interactive Systems in Practice*. ResearchGate; unknown. https://www.researchgate.net/publication/266897135_User-Centred_Systems_Design_-_Designing_Usable_Interactive_Systems_in_Practice

Huang, Z., & Benyoucef, M. (2022). An Empirical Study of Mobile Application Usability: A Unified Hierarchical approach. *International Journal of Human-Computer Interaction*, 39(13), 2624–2643. <https://doi.org/10.1080/10447318.2022.2082021>

Boehm, B., & Basili, V. (2005). Software defect Reduction Top-10 list. In *Springer eBooks* (pp. 426–431). https://doi.org/10.1007/3-540-27662-9_26

Cai, Y., & Kazman, R. (2023). Software design analysis and technical debt management based on design rule theory. *Information and Software Technology*, 164, 107322. <https://doi.org/10.1016/j.infsof.2023.107322>

Meniar, C., Opalvens, F., & Hallé, S. (2017). Runtime verification of user interface guidelines in mobile devices. In *Lecture notes in computer science* (pp. 410–415). https://doi.org/10.1007/978-3-319-67531-2_27

Ballantyne, M., Jha, A., Jacobsen, A., Hawker, J. S., & El-Glaly, Y. N. (2018). Study of Accessibility Guidelines of Mobile Applications. *Proceedings of the 17th International Conference on Mobile and Ubiquitous Multimedia - MUM 2018*. <https://doi.org/10.1145/3282894.3282921>

Laokitpanich, J., & Senivongse, T. (2022). *Heuristic-Based Usability Evaluation Tool for Android Applications for Senior Citizens*, 83–88. <https://doi.org/10.1145/3578997.3579011>

Gada, T. & Chudasama, S. (2024). The Role of User Experience in Effective Product Design Exercises Strategies for Incorporating User-Centric Approaches and Data Analysis with Business Intelligence. *International Research Journal of Modernization in Engineering Technology and Science*. <https://doi.org/10.56726/irjmets56245>

Padmasiri, P. *et al.* (2023) 'AI-Driven User Experience Design: Exploring Innovations and Challenges in Delivering Tailored User Experiences,' *2023 8th International Conference on Information Technology Research (ICITR)*, pp. 1–6. <https://doi.org/10.1109/icitr61062.2023.10382802>.

Khan, A. *et al.* (2025) 'Beyond Automation: How Designers Perceive AI as a Creative Partner in the Divergent Thinking Stages of UI/UX Design,' *CHI '25: Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pp. 1–12. <https://doi.org/10.1145/3706598.3713500>.

Lecaros, A. *et al.* (2021) 'Challenges and Opportunities on the Application of Heuristic Evaluations: A Systematic Literature review,' in *Lecture notes in computer science*, pp. 242–261. https://doi.org/10.1007/978-3-030-78221-4_17.

Takaffoli, M. *et al.* (2024) 'Generative AI in User Experience design and Research: How do UX practitioners, teams, and companies use GenAI in industry?,' *Designing Interactive Systems Conference*, pp. 1579–1593. <https://doi.org/10.1145/3643834.3660720>.

Wu, J. *et al.* (2024) 'Visual prompting in multimodal large Language models: a survey,' *arXiv (Cornell University)* [Preprint]. <https://doi.org/10.48550/arxiv.2409.15310>.

Rotaru, O.A. *et al.* (2020) 'Standardised questionnaires in usability evaluation. Applying standardised usability questionnaires in digital products evaluation,' in *Communications in computer and information science*, pp. 39–48. https://doi.org/10.1007/978-3-030-59506-7_4.

Xiong, H. *et al.* (2024) 'Converging Paradigms: The synergy of symbolic and connectionist AI in LLM-Empowered Autonomous agents,' *arXiv (Cornell University)* [Preprint]. <https://doi.org/10.48550/arxiv.2407.08516>.

Iniesto, F. *et al.* (2022) 'Creating ‘a Simple Conversation’: Designing a Conversational User Interface to Improve the Experience of Accessing Support for Study,' *ACM Transactions on Accessible Computing*, 16(1), pp. 1–29. <https://doi.org/10.1145/3568166>.

Chen, X. *et al.* (2024) 'Learning A Low-Level Vision Generalist via Visual Task Prompt,' *MM '24: Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 2671–2680. <https://doi.org/10.1145/3664647.3681621>.

W3C. (2024, December 12). *Web content accessibility guidelines (WCAG) 2.2*. [Www.w3.org; W3C. https://www.w3.org/TR/WCAG22/](https://www.w3.org/TR/WCAG22/)

Canny, J. (1986). A Computational Approach to Edge Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6), 679–698. <https://doi.org/10.1109/tpami.1986.4767851>

Beck, K., et al. (2001) The Agile Manifesto. Agile Alliance. - References - Scientific Research Publishing. (n.d.). Wwww.scirp.org.
<https://www.scirp.org/reference/referencespapers?referenceid=1360087>

Nielsen, J. (1994, April 24). 10 Heuristics for User Interface Design. Nielsen Norman Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>

8. Appendices

8.1. Gantt Chart

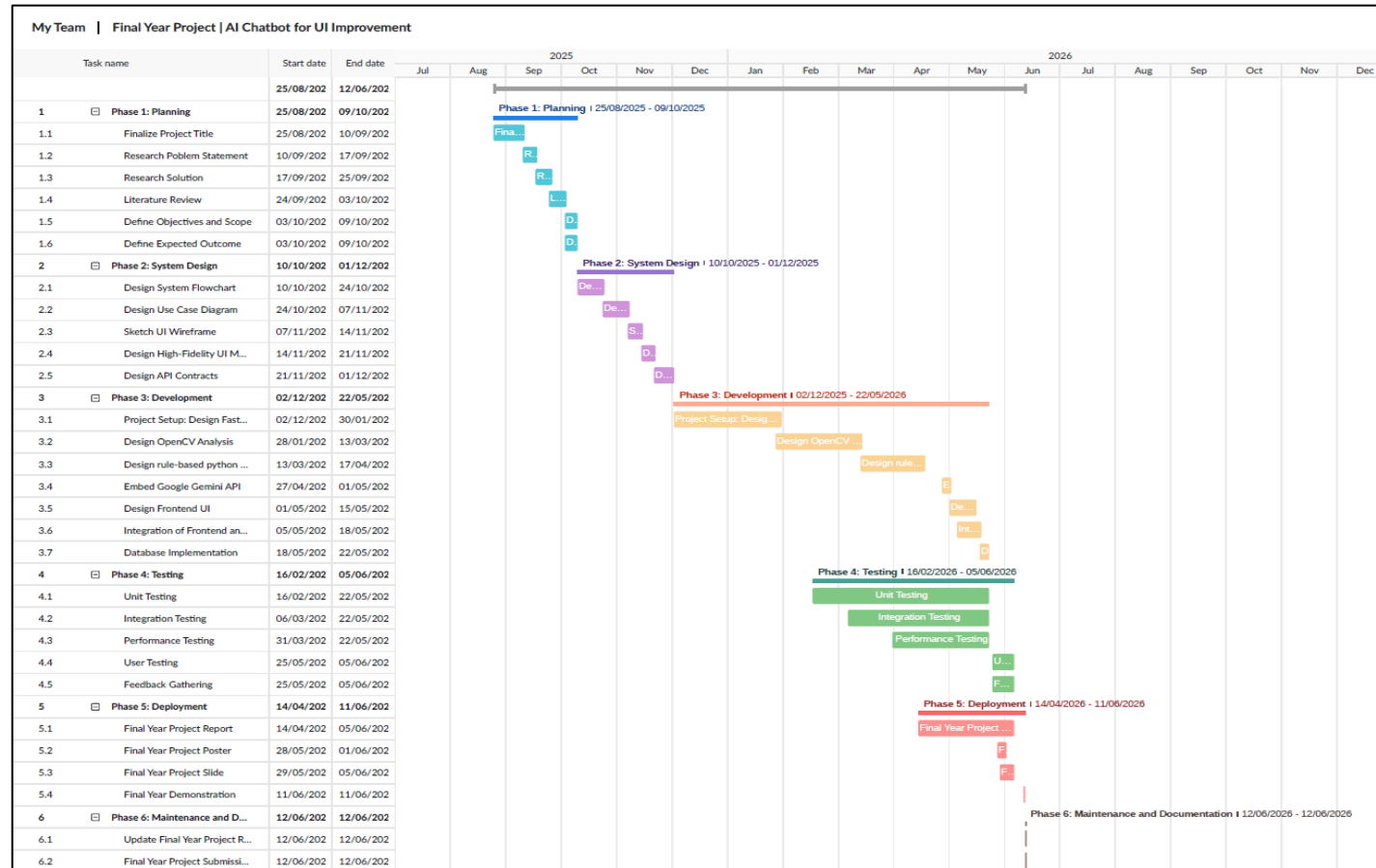


Figure 8.1.1 Project Timeline

8.2. User Test Case

6COM2017 - ARTIFICIAL INTELLIGENCE PROJECT								
USER TEST CASE								
PROJECT TITLE		AI Chatbot for UI Improvement						
Test Case ID		1						
Test User Full Name		Mr. Abdul Arriff Abdul Jalal						
Test User's Background								
a) Area of Specialization		Artificial Intelligence						
b) Job Position		Lecturer						
c) Industry		Education Industry_IT						
d) Age		42						
STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	System Launch	Verify application startup	User opens the UI Audit AI system	N/A	The application loads successfully and displays the welcome screen with chat interface.	Running without error	PASS	
2	Text-Based UX Consultation	Submit UX/UI question	User enters a UX-related question and clicks Send	"How can I improve my dashboard design?"	The system sends the query to the AI model and displays a relevant UX/UI response.	Running without error	PASS	
3	Empty Input Validation	Prevent invalid submissions	User clicks Send without entering text or uploading an image	Empty input	The system rejects the request and displays an error message requesting a question or image.	Running without error	PASS	
4	Screenshot Upload	Upload screenshot for analysis	User selects an image from local storage	dashboard_ui.png	1. The selected image is uploaded successfully 2. The selected image is displayed in the preview area.	Running without error Running without error	PASS PASS	
5	Clipboard Screenshot Upload	Upload image via paste function	User pastes an image using Ctrl + V	Screenshot from clipboard	The pasted image is detected and displayed in the preview area.	Running without error	PASS	
6	Image Preview Management	Preview and remove uploaded image	User uploads an image and clicks the Remove button	dashboard_ui.png	The image preview is removed and the upload area is reset.	Running without error	PASS	
7	UI Analysis Processing	Analyze screenshot using OpenCV and AI	User uploads a screenshot and submits analysis request	dashboard_ui.png	The system successfully performs OpenCV analysis and extracts UI metrics such as whitespace ratio, small elements, and contrast issues.	Running without error	PASS	
8	AI Audit Report Generation	Generate usability findings	User submits screenshot for audit	dashboard_ui.png	The system generates a structured audit report containing findings, severity levels, and practical recommendations.	Running without error	PASS	
9	Follow-up Conversation	Follow-up Conversation	User asks a follow-up question after receiving an audit report	"Can you explain the contrast issue?"	The system retains previous conversation context and provides a relevant follow-up response.	Running without error	PASS	
10	Audit Record Storage	Save audit results	User completes an audit session	dashboard_ui.png	The audit report and analysis data are stored successfully in the database.	Running without error	PASS	
11	System Error Handling	Handle API/server failures	Gemini API is overloaded or backend request fails	Normal image or query during high traffic	The system displays an error message such as "Backend is not responding" or "Please try again later" without crashing, and allows the user to retry.	Running without error	PASS	

Figure 8.2.1 User Test Case 1

6COM2017 - ARTIFICIAL INTELLIGENCE PROJECT								
USER TEST CASE								
PROJECT TITLE		AI Chatbot for UI Improvement						
Test Case ID		2						
Test User Full Name		Mr. Uzair Izzuddin Bin Mohd Zain						
Test User's Background								
a) Area of Specialization		Data Science						
b) Job Position		Lecturer						
c) Industry		Education Industry_IT						
d) Age		35						
STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	System Launch	Verify application startup	User opens the UI Audit AI system	N/A	The application loads successfully and displays the welcome screen with chat interface.	Running without error	PASS	
2	Text-Based UX Consultation	Submit UX/UI question	User enters a UX-related question and clicks Send	"How can I improve my dashboard design?"	The system sends the query to the AI model and displays a relevant UX/UI response.	Running without error	PASS	
3	Empty Input Validation	Prevent invalid submissions	User clicks Send without entering text or uploading an image	Empty input	The system rejects the request and displays an error message requesting a question or image.	Running without error	PASS	
4	Screenshot Upload	Upload screenshot for analysis	User selects an image from local storage	dashboard_ui.png	1. The selected image is uploaded successfully	Running without error	PASS	
					2. The selected image is displayed in the preview area.	Running without error	PASS	
5	Clipboard Screenshot Upload	Upload image via paste function	User pastes an image using Ctrl + V	Screenshot from clipboard	The pasted image is detected and displayed in the preview area.	Running without error	PASS	
6	Image Preview Management	Preview and remove uploaded image	User uploads an image and clicks the Remove button	dashboard_ui.png	The image preview is removed and the upload area is reset.	Running without error	PASS	
7	UI Analysis Processing	Analyze screenshot using OpenCV and AI	User uploads a screenshot and submits analysis request	dashboard_ui.png	The system successfully performs OpenCV analysis and extracts UI metrics such as whitespace ratio, small elements, and contrast issues.	Running without error	PASS	
8	AI Audit Report Generation	Generate usability findings	User submits screenshot for audit	dashboard_ui.png	The system generates a structured audit report containing findings, severity levels, and practical recommendations.	Running without error	PASS	
9	Follow-up Conversation	Follow-up Conversation	User asks a follow-up question after receiving an audit report	"Can you explain the contrast issue?"	The system retains previous conversation context and provides a relevant follow-up response.	Running without error	PASS	
10	Audit Record Storage	Save audit results	User completes an audit session	dashboard_ui.png	The audit report and analysis data are stored successfully in the database.	Running without error	PASS	
11	System Error Handling	Handle API/server failures	Gemini API is overloaded or backend request fails	Normal image or query during high traffic	The system displays an error message such as "Backend is not responding" or "Please try again later" without crashing, and allows the user to retry.	Running without error	PASS	

Figure 8.2.2 User Test Case 2

6COM2017 - ARTIFICIAL INTELLIGENCE PROJECT								
USER TEST CASE								
PROJECT TITLE		AI Chatbot for UI Improvement						
Test Case ID		3						
Test User Full Name		Ms. Nur Diana Madinah Binti Ab Hadi						
Test User's Background								
	a) Area of Specialization	Computer Science						
	b) Job Position	Lecturer						
	c) Industry	Education Industry_IT						
	d) Age	32						
STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	System Launch	Verify application startup	User opens the UI Audit AI system	N/A	The application loads successfully and displays the welcome screen with chat interface.	Running without error	PASS	
2	Text-Based UX Consultation	Submit UX/UI question	User enters a UX-related question and clicks Send	"How can I improve my dashboard design?"	The system sends the query to the AI model and displays a relevant UX/UI response.	Running without error	PASS	
3	Empty Input Validation	Prevent invalid submissions	User clicks Send without entering text or uploading an image	Empty input	The system rejects the request and displays an error message requesting a question or image.	Running without error	PASS	
4	Screenshot Upload	Upload screenshot for analysis	User selects an image from local storage	dashboard_ui.png	1. The selected image is uploaded successfully	Running without error	PASS	
					2. The selected image is displayed in the preview area.	Running without error	PASS	
5	Clipboard Screenshot Upload	Upload image via paste function	User pastes an image using Ctrl + V	Screenshot from clipboard	The pasted image is detected and displayed in the preview area.	Running without error	PASS	
6	Image Preview Management	Preview and remove uploaded image	User uploads an image and clicks the Remove button	dashboard_ui.png	The image preview is removed and the upload area is reset.	Running without error	PASS	
7	UI Analysis Processing	Analyze screenshot using OpenCV and AI	User uploads a screenshot and submits analysis request	dashboard_ui.png	The system successfully performs OpenCV analysis and extracts UI metrics such as whitespace ratio, small elements, and contrast issues.	Running without error	PASS	
8	AI Audit Report Generation	Generate usability findings	User submits screenshot for audit	dashboard_ui.png	The system generates a structured audit report containing findings, severity levels, and practical recommendations.	Running without error	PASS	
9	Follow-up Conversation	Follow-up Conversation	User asks a follow-up question after receiving an audit report	"Can you explain the contrast issue?"	The system retains previous conversation context and provides a relevant follow-up response.	Running without error	PASS	
10	Audit Record Storage	Save audit results	User completes an audit session	dashboard_ui.png	The audit report and analysis data are stored successfully in the database.	Running without error	PASS	
11	System Error Handling	Handle API/server failures	Gemini API is overloaded or backend request fails	Normal image or query during high traffic	The system displays an error message such as "Backend is not responding" or "Please try again later" without crashing, and allows the user to retry.	Running without error	PASS	

Figure 8.2.3 User Test Case 3

6COM2017 - ARTIFICIAL INTELLIGENCE PROJECT								
USER TEST CASE								
PROJECT TITLE		AI Chatbot for UI Improvement						
Test Case ID		4						
Test User Full Name		Ts. Emil Anthony Arputham						
Test User's Background								
a) Area of Specialization		Cybersecurity						
b) Job Position		Lecturer						
c) Industry		Education Industry_IT						
d) Age		37						
STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	System Launch	Verify application startup	User opens the UI Audit AI system	N/A	The application loads successfully and displays the welcome screen with chat interface.	Running without error	PASS	
2	Text-Based UX Consultation	Submit UX/UI question	User enters a UX-related question and clicks Send	"How can I improve my dashboard design?"	The system sends the query to the AI model and displays a relevant UX/UI response.	Running without error	PASS	
3	Empty Input Validation	Prevent invalid submissions	User clicks Send without entering text or uploading an image	Empty input	The system rejects the request and displays an error message requesting a question or image.	Running without error	PASS	
4	Screenshot Upload	Upload screenshot for analysis	User selects an image from local storage	dashboard_ui.png	1. The selected image is uploaded successfully	Running without error	PASS	
					2. The selected image is displayed in the preview area.	Running without error	PASS	
5	Clipboard Screenshot Upload	Upload image via paste function	User pastes an image using Ctrl + V	Screenshot from clipboard	The pasted image is detected and displayed in the preview area.	Running without error	PASS	
6	Image Preview Management	Preview and remove uploaded image	User uploads an image and clicks the Remove button	dashboard_ui.png	The image preview is removed and the upload area is reset.	Running without error	PASS	
7	UI Analysis Processing	Analyze screenshot using OpenCV and AI	User uploads a screenshot and submits analysis request	dashboard_ui.png	The system successfully performs OpenCV analysis and extracts UI metrics such as whitespace ratio, small elements, and contrast issues.	Running without error	PASS	
8	AI Audit Report Generation	Generate usability findings	User submits screenshot for audit	dashboard_ui.png	The system generates a structured audit report containing findings, severity levels, and practical recommendations.	Running without error	PASS	
9	Follow-up Conversation	Follow-up Conversation	User asks a follow-up question after receiving an audit report	"Can you explain the contrast issue?"	The system retains previous conversation context and provides a relevant follow-up response.	Running without error	PASS	
10	Audit Record Storage	Save audit results	User completes an audit session	dashboard_ui.png	The audit report and analysis data are stored successfully in the database.	Running without error	PASS	
11	System Error Handling	Handle API/server failures	Gemini API is overloaded or backend request fails	Normal image or query during high traffic	The system displays an error message such as "Backend is not responding" or "Please try again later" without crashing, and allows the user to retry.	Running without error	PASS	

Figure 8.2.4 User Test Case 4

6COM2017 - ARTIFICIAL INTELLIGENCE PROJECT								
USER TEST CASE								
PROJECT TITLE		AI Chatbot for UI Improvement						
Test Case ID		5						
Test User Full Name		Chew Zi Hau						
Test User's Background								
a) Area of Specialization		Cybersecurity						
b) Job Position		Student						
c) Industry								
d) Age		24						
STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	System Launch	Verify application startup	User opens the UI Audit AI system	N/A	The application loads successfully and displays the welcome screen with chat interface.	Running without error	PASS	
2	Text-Based UX Consultation	Submit UX/UI question	User enters a UX-related question and clicks Send	"How can I improve my dashboard design?"	The system sends the query to the AI model and displays a relevant UX/UI response.	Running without error	PASS	
3	Empty Input Validation	Prevent invalid submissions	User clicks Send without entering text or uploading an image	Empty input	The system rejects the request and displays an error message requesting a question or image.	Running without error	PASS	
4	Screenshot Upload	Upload screenshot for analysis	User selects an image from local storage	dashboard_ui.png	1. The selected image is uploaded successfully. 2. The selected image is displayed in the preview area.	Running without error Running without error	PASS PASS	
5	Clipboard Screenshot Upload	Upload image via paste function	User pastes an image using Ctrl + V	Screenshot from clipboard	The pasted image is detected and displayed in the preview area.	Running without error	PASS	
6	Image Preview Management	Preview and remove uploaded image	User uploads an image and clicks the Remove button	dashboard_ui.png	The image preview is removed and the upload area is reset.	Running without error	PASS	
7	UI Analysis Processing	Analyze screenshot using OpenCV and AI	User uploads a screenshot and submits analysis request	dashboard_ui.png	The system successfully performs OpenCV analysis and extracts UI metrics such as whitespace ratio, small elements, and contrast issues.	Running without error	PASS	
8	AI Audit Report Generation	Generate usability findings	User submits screenshot for audit	dashboard_ui.png	The system generates a structured audit report containing findings, severity levels, and practical recommendations.	Running without error	PASS	
9	Follow-up Conversation	Follow-up Conversation	User asks a follow-up question after receiving an audit report	"Can you explain the contrast issue?"	The system retains previous conversation context and provides a relevant follow-up response.	Running without error	PASS	
10	Audit Record Storage	Save audit results	User completes an audit session	dashboard_ui.png	The audit report and analysis data are stored successfully in the database.	Running without error	PASS	
11	System Error Handling	Handle API/server failures	Gemini API is overloaded or backend request fails	Normal image or query during high traffic	The system displays an error message such as "Backend is not responding" or "Please try again later" without crashing, and allows the user to retry.	Running without error	PASS	

Figure 8.2.5 User Test Case 5

6COM2017 - ARTIFICIAL INTELLIGENCE PROJECT								
USER TEST CASE								
PROJECT TITLE		AI Chatbot for UI Improvement						
Test Case ID		6						
Test User Full Name		Dayaani A/P Nathan						
Test User's Background								
a) Area of Specialization		Cybersecurity						
b) Job Position		Student						
c) Industry								
d) Age		24						
STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	System Launch	Verify application startup	User opens the UI Audit AI system	N/A	The application loads successfully and displays the welcome screen with chat interface.	Running without error	PASS	
2	Text-Based UX Consultation	Submit UX/UI question	User enters a UX-related question and clicks Send	"How can I improve my dashboard design?"	The system sends the query to the AI model and displays a relevant UX/UI response.	Running without error	PASS	
3	Empty Input Validation	Prevent invalid submissions	User clicks Send without entering text or uploading an image	Empty input	The system rejects the request and displays an error message requesting a question or image.	Running without error	PASS	
4	Screenshot Upload	Upload screenshot for analysis	User selects an image from local storage	dashboard_ui.png	1. The selected image is uploaded successfully. 2. The selected image is displayed in the preview area.	Running without error Running without error	PASS PASS	
5	Clipboard Screenshot Upload	Upload image via paste function	User pastes an image using Ctrl + V	Screenshot from clipboard	The pasted image is detected and displayed in the preview area.	Running without error	PASS	
6	Image Preview Management	Preview and remove uploaded image	User uploads an image and clicks the Remove button	dashboard_ui.png	The image preview is removed and the upload area is reset.	Running without error	PASS	
7	UI Analysis Processing	Analyze screenshot using OpenCV and AI	User uploads a screenshot and submits analysis request	dashboard_ui.png	The system successfully performs OpenCV analysis and extracts UI metrics such as whitespace ratio, small elements, and contrast issues.	Running without error	PASS	
8	AI Audit Report Generation	Generate usability findings	User submits screenshot for audit	dashboard_ui.png	The system generates a structured audit report containing findings, severity levels, and practical recommendations.	Running without error	PASS	
9	Follow-up Conversation	Follow-up Conversation	User asks a follow-up question after receiving an audit report	"Can you explain the contrast issue?"	The system retains previous conversation context and provides a relevant follow-up response.	Running without error	PASS	
10	Audit Record Storage	Save audit results	User completes an audit session	dashboard_ui.png	The audit report and analysis data are stored successfully in the database.	Running without error	PASS	
11	System Error Handling	Handle API/server failures	Gemini API is overloaded or backend request fails	Normal image or query during high traffic	The system displays an error message such as "Backend is not responding" or "Please try again later" without crashing and allows the user to retry.	Running without error	PASS	

Figure 8.2.6 User Test Case 6

6COM2017 - ARTIFICIAL INTELLIGENCE PROJECT								
USER TEST CASE								
PROJECT TITLE		AI Chatbot for UI Improvement						
Test Case ID		7						
Test User Full Name		Tan Jia Jian						
Test User's Background								
a) Area of Specialization		Cybersecurity						
b) Job Position		Student						
c) Industry								
d) Age		24						
STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	System Launch	Verify application startup	User opens the UI Audit AI system	N/A	The application loads successfully and displays the welcome screen with chat interface.	Running without error	PASS	
2	Text-Based UX Consultation	Submit UX/UI question	User enters a UX-related question and clicks Send	"How can I improve my dashboard design?"	The system sends the query to the AI model and displays a relevant UX/UI response.	Running without error	PASS	
3	Empty Input Validation	Prevent invalid submissions	User clicks Send without entering text or uploading an image	Empty input	The system rejects the request and displays an error message requesting a question or image.	Running without error	PASS	
4	Screenshot Upload	Upload screenshot for analysis	User selects an image from local storage	dashboard_ui.png	1. The selected image is uploaded successfully	Running without error	PASS	
					2. The selected image is displayed in the preview area.	Running without error	PASS	
5	Clipboard Screenshot Upload	Upload image via paste function	User pastes an image using Ctrl + V	Screenshot from clipboard	The pasted image is detected and displayed in the preview area.	Running without error	PASS	
6	Image Preview Management	Preview and remove uploaded image	User uploads an image and clicks the Remove button	dashboard_ui.png	The image preview is removed and the upload area is reset.	Running without error	PASS	
7	UI Analysis Processing	Analyze screenshot using OpenCV and AI	User uploads a screenshot and submits analysis request	dashboard_ui.png	The system successfully performs OpenCV analysis and extracts UI metrics such as whitespace ratio, small elements, and contrast issues.	Running without error	PASS	
8	AI Audit Report Generation	Generate usability findings	User submits screenshot for audit	dashboard_ui.png	The system generates a structured audit report containing findings, severity levels, and practical recommendations.	Running without error	PASS	
9	Follow-up Conversation	Follow-up Conversation	User asks a follow-up question after receiving an audit report	"Can you explain the contrast issue?"	The system retains previous conversation context and provides a relevant follow-up response.	Running without error	PASS	
10	Audit Record Storage	Save audit results	User completes an audit session	dashboard_ui.png	The audit report and analysis data are stored successfully in the database.	Running without error	PASS	
11	System Error Handling	Handle API/server failures	Gemini API is overloaded or backend request fails	Normal image or query during high traffic	The system displays an error message such as "Backend is not responding" or "Please try again later" without crashing and allows the user to retry.	Running without error	PASS	

Figure 8.2.7 User Test Case 7

6COM2017 - ARTIFICIAL INTELLIGENCE PROJECT								
USER TEST CASE								
PROJECT TITLE		AI Chatbot for UI Improvement						
Test Case ID		8						
Test User Full Name		Lay Xian Zu						
Test User's Background								
a) Area of Specialization		Information Technology						
b) Job Position		Sales						
c) Industry								
d) Age		25						
STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	System Launch	Verify application startup	User opens the UI Audit AI system	N/A	The application loads successfully and displays the welcome screen with chat interface.	Running without error	PASS	
2	Text-Based UX Consultation	Submit UX/UI question	User enters a UX-related question and clicks Send	"How can I improve my dashboard design?"	The system sends the query to the AI model and displays a relevant UX/UI response.	Running without error	PASS	
3	Empty Input Validation	Prevent invalid submissions	User clicks Send without entering text or uploading an image	Empty input	The system rejects the request and displays an error message requesting a question or image.	Running without error	PASS	
4	Screenshot Upload	Upload screenshot for analysis	User selects an image from local storage	dashboard_ui.png	1. The selected image is uploaded successfully. 2. The selected image is displayed in the preview area.	Running without error Running without error	PASS PASS	
5	Clipboard Screenshot Upload	Upload image via paste function	User pastes an image using Ctrl + V	Screenshot from clipboard	The pasted image is detected and displayed in the preview area.	Running without error	PASS	
6	Image Preview Management	Preview and remove uploaded image	User uploads an image and clicks the Remove button	dashboard_ui.png	The image preview is removed and the upload area is reset.	Running without error	PASS	
7	UI Analysis Processing	Analyze screenshot using OpenCV and AI	User uploads a screenshot and submits analysis request	dashboard_ui.png	The system successfully performs OpenCV analysis and extracts UI metrics such as whitespace ratio, small elements, and contrast issues.	Running without error	PASS	
8	AI Audit Report Generation	Generate usability findings	User submits screenshot for audit	dashboard_ui.png	The system generates a structured audit report containing findings, severity levels, and practical recommendations.	Running without error	PASS	
9	Follow-up Conversation	Follow-up Conversation	User asks a follow-up question after receiving an audit report	"Can you explain the contrast issue?"	The system retains previous conversation context and provides a relevant follow-up response.	Running without error	PASS	
10	Audit Record Storage	Save audit results	User completes an audit session	dashboard_ui.png	The audit report and analysis data are stored successfully in the database.	Running without error	PASS	
11	System Error Handling	Handle API/server failures	Gemini API is overloaded or backend request fails	Normal image or query during high traffic	The system displays an error message such as "Backend is not responding" or "Please try again later" without crashing, and allows the user to retry.	Running without error	PASS	

Figure 8.2.8 User Test Case 8

6COM2017 - ARTIFICIAL INTELLIGENCE PROJECT								
USER TEST CASE								
PROJECT TITLE		AI Chatbot for UI Improvement						
Test Case ID		9						
Test User Full Name		Weedon						
Test User's Background								
	a) Area of Specialization	Mechanical Engineering						
	b) Job Position	Product Manager						
	c) Industry	IT Business						
	d) Age	29						
STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	System Launch	Verify application startup	User opens the UI Audit AI system	N/A	The application loads successfully and displays the welcome screen with chat interface.	Running without error	PASS	
2	Text-Based UX Consultation	Submit UX/UI question	User enters a UX-related question and clicks Send	"How can I improve my dashboard design?"	The system sends the query to the AI model and displays a relevant UX/UI response.	Running without error	PASS	
3	Empty Input Validation	Prevent invalid submissions	User clicks Send without entering text or uploading an image	Empty input	The system rejects the request and displays an error message requesting a question or image.	Running without error	PASS	
4	Screenshot Upload	Upload screenshot for analysis	User selects an image from local storage	dashboard_ui.png	1. The selected image is uploaded successfully	Running without error	PASS	
					2. The selected image is displayed in the preview area.	Running without error	PASS	
5	Clipboard Screenshot Upload	Upload image via paste function	User pastes an image using Ctrl + V	Screenshot from clipboard	The pasted image is detected and displayed in the preview area.	Running without error	PASS	
6	Image Preview Management	Preview and remove uploaded image	User uploads an image and clicks the Remove button	dashboard_ui.png	The image preview is removed and the upload area is reset.	Running without error	PASS	
7	UI Analysis Processing	Analyze screenshot using OpenCV and AI	User uploads a screenshot and submits analysis request	dashboard_ui.png	The system successfully performs OpenCV analysis and extracts UI metrics such as whitespace ratio, small elements, and contrast issues.	Running without error	PASS	
8	AI Audit Report Generation	Generate usability findings	User submits screenshot for audit	dashboard_ui.png	The system generates a structured audit report containing findings, severity levels, and practical recommendations.	Running without error	PASS	
9	Follow-up Conversation	Follow-up Conversation	User asks a follow-up question after receiving an audit report	"Can you explain the contrast issue?"	The system retains previous conversation context and provides a relevant follow-up response.	Running without error	PASS	
10	Audit Record Storage	Save audit results	User completes an audit session	dashboard_ui.png	The audit report and analysis data are stored successfully in the database.	Running without error	PASS	
11	System Error Handling	Handle API/server failures	Gemini API is overloaded or backend request fails	Normal image or query during high traffic	The system displays an error message such as "Backend is not responding" or "Please try again later" without crashing, and allows the user to retry.	Running without error	PASS	

Figure 8.2.9 User Test Case 9

6COM2017 - ARTIFICIAL INTELLIGENCE PROJECT								
USER TEST CASE								
PROJECT TITLE		AI Chatbot for UI Improvement						
Test Case ID		10						
Test User Full Name		Tan Kian Ho						
Test User's Background								
	a) Area of Specialization	Computer Science						
	b) Job Position	Mobile Developer						
	c) Industry	IT Business						
	d) Age	25						
STEPS	USE CASE	FUNCTION	SCENARIOS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS	REMARKS
1	System Launch	Verify application startup	User opens the UI Audit AI system	N/A	The application loads successfully and displays the welcome screen with chat interface.	Running without error	PASS	
2	Text-Based UX Consultation	Submit UX/UI question	User enters a UX-related question and clicks Send	"How can I improve my dashboard design?"	The system sends the query to the AI model and displays a relevant UX/UI response.	Running without error	PASS	
3	Empty Input Validation	Prevent invalid submissions	User clicks Send without entering text or uploading an image	Empty input	The system rejects the request and displays an error message requesting a question or image.	Running without error	PASS	
4	Screenshot Upload	Upload screenshot for analysis	User selects an image from local storage	dashboard_ui.png	1. The selected image is uploaded successfully	Running without error	PASS	
					2. The selected image is displayed in the preview area.	Running without error	PASS	
5	Clipboard Screenshot Upload	Upload image via paste function	User pastes an image using Ctrl + V	Screenshot from clipboard	The pasted image is detected and displayed in the preview area.	Running without error	PASS	
6	Image Preview Management	Preview and remove uploaded image	User uploads an image and clicks the Remove button	dashboard_ui.png	The image preview is removed and the upload area is reset.	Running without error	PASS	
7	UI Analysis Processing	Analyze screenshot using OpenCV and AI	User uploads a screenshot and submits analysis request	dashboard_ui.png	The system successfully performs OpenCV analysis and extracts UI metrics such as whitespace ratio, small elements, and contrast issues.	Running without error	PASS	
8	AI Audit Report Generation	Generate usability findings	User submits screenshot for audit	dashboard_ui.png	The system generates a structured audit report containing findings, severity levels, and practical recommendations.	Running without error	PASS	
9	Follow-up Conversation	Follow-up Conversation	User asks a follow-up question after receiving an audit report	"Can you explain the contrast issue?"	The system retains previous conversation context and provides a relevant follow-up response.	Running without error	PASS	
10	Audit Record Storage	Save audit results	User completes an audit session	dashboard_ui.png	The audit report and analysis data are stored successfully in the database.	Running without error	PASS	
11	System Error Handling	Handle API/server failures	Gemini API is overloaded or backend request fails	Normal image or query during high traffic	The system displays an error message such as "Backend is not responding" or "Please try again later" without crashing, and allows the user to retry.	Running without error	PASS	

Figure 8.2.10 User Test Case 10

8.3. Code Implementation

FastAPI_Backend.py

```
import os
import logging
import uvicorn
from fastapi import FastAPI, UploadFile, File, HTTPException, Form
from fastapi.middleware.cors import CORSMiddleware
from google import genai
from dotenv import load_dotenv
from cv_analysis import perform_cv_audit
from typing import Optional
import rules
from database import db_helper
import time

# Setup logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

load_dotenv()

API_KEY = os.getenv("GEMINI_API_KEY")
if not API_KEY:
    raise RuntimeError("GEMINI_API_KEY not found in .env file.")

app = FastAPI(title="AI UI Auditor", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"]
)

client = genai.Client(api_key=API_KEY)

def classify_ui(cv):
    whitespace = cv.get("whitespace_ratio", 0)
    small_elements = cv.get("small_elements_found", 0)
    contrast = cv.get("contrast_issues_found", 0)

    # Heuristic scoring
    score = 100

    # Penalties
    if whitespace < 15:
        score -= 20
    elif whitespace > 80:
```

```

score -= 10

if small_elements > 50:
    score -= 25
elif small_elements > 20:
    score -= 10

if contrast > 10:
    score -= 25
elif contrast > 5:
    score -= 10

# Convert score into category
if score >= 80:
    return "GOOD", score
elif score >= 50:
    return "AVERAGE", score
else:
    return "POOR", score

# --- Security Guardrail Definition ---
SYSTEM_INSTRUCTION = f"""
You are a Senior UX/UI and Accessibility Auditor.

PERSONALITY:
- Professional, insightful, and slightly witty.
- Use emojis sparingly to keep it modern.
- DO NOT start every response with a greeting. Only greet the user if they
greet you first.

STRICT DESIGN FOCUS:
- Your core job is to analyze UI based on WCAG and Nielsen Heuristics.
- If a user asks something totally unrelated (sports, recipes, etc.), politely
decline:
    "I'm specialized in UX/UI auditing. I can't help with that, but I'd love to
audit a screenshot or answer a design question!"
- Always reference heuristics by their short name in parentheses, e.g.,
(Visibility) or (Aesthetic).

TECHNICAL KNOWLEDGE BASE (Use these for your audits):
- Target Size: {rules.MIN_TOUCH_TARGET_AA}px (AA) or
{rules.MIN_TOUCH_TARGET_AAA}px (AAA).
- Contrast: {rules.MIN_LARGE_TEXT_CONTRAST}:1 for normal text.
- Heuristics to use: {rules.HEURISTICS}
"""

def build_audit_prompt(cv_data, ui_quality, ui_score, user_query=None):
    # 1. Get the strict rules from rules.py
    audit_rules = rules.get_rules_string()

```

2. Build the factual data string from OpenCV

analysis_data = f"""

IMAGE ANALYSIS DATA (FACTS):

- UI Quality Level: {ui_quality}
- UI Score: {ui_score}/100

- Whitespace Ratio: {cv_data.get('whitespace_ratio', 0)}%
- Small Interactive Elements: {cv_data.get('small_elements_found', 0)}
- Contrast Issues: {cv_data.get('contrast_issues_found', 0)}

"""

3. Create the Base Prompt

base_prompt = f"""

SYSTEM ROLE: You are an expert Technical UI Auditor.

{audit_rules}

{analysis_data}

TASK:

You are a professional UI/UX auditor.

First, consider the UI QUALITY LEVEL:

- If GOOD → emphasize strengths, only minor suggestions.
- If AVERAGE → balanced feedback.
- If POOR → highlight real usability issues.

IMPORTANT RULE:

Do NOT force problems. If UI is good, you may return fewer than 3 issues or even state "No major usability issues detected".

"""

4. Handle the User Query

if user_query:

base_prompt += f"\nUSER CONTEXT/QUESTION:

{user_query}\nINSTRUCTION: Address the user's question specifically while following the output constraints below."

5. Final Output Constraint

constraint = """

\nOUTPUT FORMAT (STRICT - MUST FOLLOW EXACTLY):

You must output up to 3 findings only.

Each finding MUST follow this exact format:

Point X: Title

****Finding**:**

```

<text>

**Severity**:
<Critical | Major | Minor>

**Practical Fix**:
<text>

---

Rules:
- If UI is GOOD, you may output 0–2 findings.
- If UI is AVERAGE, output 2–3 findings.
- If UI is POOR, output up to 3 findings.
- You may include strengths if relevant.
- "Finding:", "Severity:", and "Practical Fix:" MUST be on separate lines.
- Do NOT combine fields on the same line.
- Do NOT use inline formatting like "Severity: Minor Practical Fix:".
- Always add a line break after each label.
- Always separate sections using "---".
- Maximum 3 findings only.

""""

return base_prompt + constraint

@app.post("/analyze/ui-screenshot")
async def analyze_ui_screenshot(
    file: Optional[UploadFile] = File(None),
    query: Optional[str] = Form(None),
    history: Optional[str] = Form(None)
):
    total_timer_start = time.time()

    is_file_provided = file is not None and file.filename != ""
    is_query_provided = query is not None and query.strip() != ""

    if not is_file_provided and not is_query_provided:
        raise HTTPException(status_code=400, detail="Please provide a
question or an image.")

    try:
        # Configuration for the model to use the System Instruction properly
        model_id = 'gemini-2.5-flash'
        config = {"system_instruction": SYSTEM_INSTRUCTION}

        # Parse History
        import json
        past_messages = json.loads(history) if history else []

```

```

# Initialize the Chat Session with history
chat = client.chats.create(
    model=model_id,
    config=config,
    history=past_messages
)

# CASE 1: TEXT ONLY
if not is_file_provided:

    ai_timer_start = time.time()

    response = chat.send_message(query)

    ai_timer_end = time.time()

    logger.info(f"AI Response Time: {ai_timer_end -
ai_timer_start:.6f}s")

    return {"status": "success", "opencv_data": None,
"gemini_response": response.text}

# CASE 2: IMAGE (+ Optional Text)

upload_tiemr_start = time.perf_counter()

image_data = await file.read()

upload_tiemr_end = time.perf_counter()

upload_time_ms = (upload_tiemr_end - upload_tiemr_start) * 1000

if len(image_data) == 0:
    raise ValueError("The uploaded file is empty.")

# Run the Computer Vision Audit
opencv_timer_start = time.time()

cv_results, optimized_image = perform_cv_audit(image_data)

opencv_timer_end = time.time()

opencv_time = opencv_timer_end - opencv_timer_start

# UI classification (IMPORTANT)
ui_quality, ui_score = classify_ui(cv_results)

# Prepare the image part for Gemini
image_part = genai.types.Part.from_bytes(data = optimized_image,
mime_type = "image/jpeg")

```



```

        # Build the dynamic prompt using the CV data
        audit_prompt =
build_audit_prompt(cv_results,ui_quality,ui_score,query)

        ai_timer_start = time.time()

        response = chat.send_message([image_part, audit_prompt])

        ai_timer_end = time.time()

        ai_time = ai_timer_end - ai_timer_start

        db_timer_start = time.time()

        await db_helper.save_audit(
            filename=file.filename if is_file_provided else "text_query",
            query=query or "N/A",
            metrics=cv_results if is_file_provided else {"note": "text-only
query"},
            report=response.text
        )

        db_timer_end = time.time()

        db_time = db_timer_end - db_timer_start

        total_timer_end = time.time()

        total_time = total_timer_end - total_timer_start

    # Log Result
    logger.info(f"Upload Time: {upload_time_ms:.3f}s")
    logger.info(f"OpenCV Time: {opencv_time:.2f}s")
    logger.info(f"AI Time: {ai_time:.2f}s")
    logger.info(f"Database Time: {db_time:.2f}s")
    logger.info(f"Total Time: {total_time:.2f}s")

    return {
        "status": "success",
        "opencv_data": cv_results,
        "gemini_response": response.text,
    }

except Exception as e:
    logger.error(f"Analysis Error: {str(e)}")
    raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)

```



```

# Knowledge Base

# --- 1. ACCESSIBILITY (WCAG 2.2) ---
# Perceivable: Contrast
MIN_TEXT_CONTRAST = 4.5
MIN_LARGE_TEXT_CONTRAST = 3.0
MIN_UI_COMPONENT_CONTRAST = 3.0

# Operable: Touch Targets
MIN_TOUCH_TARGET_AA = 24 # CSS Pixels
MIN_TOUCH_TARGET_AAA = 44

# --- 2. USABILITY HEURISTICS (Nielsen) ---
HEURISTICS = {
    "Visibility": "Visibility of System Status: Show loading states or
progress.",
    "Match": "Match system to real world: Use natural language, avoid
jargon.",
    "Control": "User control: Provide 'Undo', 'Cancel', or 'Back' options.",
    "Consistency": "Consistency and standards: Buttons/UI must look
uniform.",
    "Error Prevention": "Prevent errors: Guard destructive actions (e.g.
Delete).",
    "Recognition": "Recognition over recall: Use clear labels and icons.",
    "Flexibility": "Flexibility: Provide shortcuts/filters for power users.",
    "Aesthetic": "Aesthetic design: Avoid clutter/minimalist whitespace.",
    "Error Recovery": "Help users recover: Messages must be plain-text and
helpful.",
    "Help": "Help: Search or help icons must be easily accessible."
}

# --- 3. SCORING & SEVERITY ---
WEIGHTS = {
    "CRITICAL": 25, # Blockers: Contrast failures, missing buttons
    "MAJOR": 15, # Friction: Small targets, high clutter
    "MINOR": 5 # Polish: Spacing, font inconsistencies
}

# --- 4. AI PROMPT BRIDGE ---
def get_rules_string():
    """Converts Python constants into a string for the AI to read."""
    rules_text = f"""
AUDIT THRESHOLDS:
- Text Contrast: Min {MIN_TEXT_CONTRAST}:1 (Large text:
{MIN_LARGE_TEXT_CONTRAST}:1)
- UI Component Contrast: Min
{MIN_UI_COMPONENT_CONTRAST}:1

```

```
- Touch Target Size: Min  
{MIN_TOUCH_TARGET_AA}x{MIN_TOUCH_TARGET_AA}px  
- Max Mobile Width: 320px
```

```
HEURISTIC DEFINITIONS:
```

```
"""
```

```
for key, val in HEURISTICS.items():
```

```
    rules_text += f"- {key}: {val}\n"
```

```
return rules_text
```

cv_analysis.py

```
import cv2
import numpy as np
import rules

def get_luminance(color_bgr):
    """Calculates relative luminance. Safely handles various input types."""
    try:
        # Standardizing to floats for precise division
        b = float(color_bgr[0]) / 255.0
        g = float(color_bgr[1]) / 255.0
        r = float(color_bgr[2]) / 255.0
    except (IndexError, TypeError, ValueError):
        return 0

    # WCAG Relative Luminance Formula: 0.2126*R + 0.7152*G + 0.0722*B
    standardized = []
    for c in [r, g, b]:
        if c <= 0.03928:
            standardized.append(c / 12.92)
        else:
            standardized.append((((c + 0.055) / 1.055) ** 2.4))

    return 0.2126 * standardized[0] + 0.7152 * standardized[1] + 0.0722 * standardized[2]

def calculate_contrast(color1, color2):
    l1 = get_luminance(color1)
    l2 = get_luminance(color2)
    return round((max(l1, l2) + 0.05) / (min(l1, l2) + 0.05), 2)

def calculate_whitespace_ratio(img_gray):
    # Canny edge detection helps identify "busy" vs "empty" areas
    edges = cv2.Canny(img_gray, 50, 150)
    kernel = np.ones((5,5), np.uint8)
    busy_areas = cv2.dilate(edges, kernel, iterations=2)
    empty_pixels = cv2.countNonZero(cv2.bitwise_not(busy_areas))
    return round((empty_pixels / (img_gray.shape[0] * img_gray.shape[1])) * 100, 2)

def perform_cv_audit(image_bytes):
    nparr = np.frombuffer(image_bytes, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if img is None: return {"error": "Invalid image"}, image_bytes

    h, w = img.shape[:2]
    if w > 1024:
        img = cv2.resize(img, (1024, int(h * (1024 / w))))
```

```

# Visual Debugging: Create a copy to draw on
debug_img = img.copy()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Detection logic
corners = [img[0,0], img[0,-1], img[-1,0], img[-1,-1]]
bg_color = np.mean(corners, axis=0)

thresh = cv2.adaptiveThreshold(gray, 255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                cv2.THRESH_BINARY_INV, 11, 2)
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

small_elements = 0
contrast_issues = 0
sampled_ratios = []

for cnt in contours:
    x, y, w_box, h_box = cv2.boundingRect(cnt)
    area = cv2.contourArea(cnt)

    # FIX: Ignore "Noise" (tiny dots) but keep "Small Targets"
    if area < 50: continue

    # 1. Touch Target Check (WCAG 2.5.5)
    if w_box < rules.MIN_TOUCH_TARGET_AA or h_box <
rules.MIN_TOUCH_TARGET_AAA:
        small_elements += 1
        # Draw a YELLOW box around small targets
        cv2.rectangle(debug_img, (x, y), (x + w_box, y + h_box), (0, 255,
255), 1)

    # 2. Contrast Check (WCAG 1.4.3)
    if w_box > 15 and h_box > 15:
        roi = img[y:y+h_box, x:x+w_box]
        avg_color = cv2.mean(roi)[:3]
        ratio = calculate_contrast(avg_color, bg_color)
        sampled_ratios.append(ratio)

        if ratio < rules.MIN_TEXT_CONTRAST:
            contrast_issues += 1
            # Draw a RED box around contrast failures
            cv2.rectangle(debug_img, (x, y), (x + w_box, y + h_box), (0, 0,
255), 2)

    whitespace = calculate_whitespace_ratio(gray)
    avg_contrast = round(sum(sampled_ratios)/len(sampled_ratios), 1) if
sampled_ratios else "N/A"

```

```
# Encode the DEBUG image so Gemini can see the boxes
_, compressed = cv2.imencode('.jpg', debug_img,
[cv2.IMWRITE_JPEG_QUALITY, 85])

return {
    "whitespace_ratio": whitespace,
    "is_cluttered": whitespace < 15,
    "small_elements_found": small_elements,
    "contrast_issues_found": contrast_issues,
    "average_contrast_ratio": avg_contrast,
}, compressed.tobytes()
```

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>AI UI Audit Chatbot</title>
  <link rel="stylesheet"
href="https://fonts.googleapis.com/css2?family=Material+Symbols+Outline
d:opsz,wght,FILL,GRAD@24,400,0..1,0" />
  <link rel="stylesheet" href="interface_css.css">
  <script
src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>

  <style>
    .material-symbols-outlined {
      font-variation-settings: 'FILL' 1;
    }

    .header-right .material-symbols-outlined {
      font-variation-settings: 'FILL' 0;
    }

    .help-dropdown .material-symbols-outlined {
      font-variation-settings: 'FILL' 0;
    }
  </style>
</head>
<body>
  <header id="top-header">
    <div class="header-content">
      <div class="header-left">
        <span class="logo-text">UI Audit AI</span>
      </div>

      <div class="header-right">
        <button class="icon-btn" title="help"
onclick="toggleDropdown(event)">
          <span id="help-icon" class="material-symbols-
outlined">help</span>
        </button>

        <div id="help-dropdown" class="dropdown-content">
          <div class="dropdown-item">
            <span class="material-symbols-outlined">help</span>
            <span>Help Center</span>
          </div>
          <div class="dropdown-item">

```



```

        <span class="material-symbols-outlined">settings</span>
        <span>Settings</span>
    </div>
</div>
</div>
</div>
</div>
</header>

<div id="app-container">
    <div id="chat-container">
        <div id="welcome-screen">
            <h1>Hi, How may I help you today?</h1>
            <p>Upload a UI screenshot for a technical audit or ask a UX
question.</p>
        </div>
    </div>

    <div id="loader"><span>Thinking...</span></div>

    <div class="input-area">
        <div class="input-container">
            <div id="preview-area"></div>
            <textarea id="user-input" placeholder="Ask about your UI
design..." rows="1" oninput="autoGrow(this)"
onkeypress="handleKeyPress(event)"></textarea>

            <div class="input-actions">
                <label for="file-input" class="icon-btn" style="cursor:pointer">
                    <span class="material-symbols-outlined">add</span>
                </label>
                <input type="file" id="file-input" style="display:none"
accept="image/*" onchange="handleFileSelect()">

                <button class="icon-btn" onclick="sendMessage()">
                    <span class="material-symbols-outlined">send</span>
                </button>
            </div>
        </div>
        <div class="disclaimer-text">AI can make mistakes. Check
important info.</div>
    </div>

    <div id="image-modal" class="modal" onclick="closeModal()">
        <img class="modal-content" id="full-image"
onclick="event.stopPropagation()">
    </div>

<script>
    let chatHistory = [];

```

```

const chatContainer = document.getElementById('chat-container');
const previewArea = document.getElementById('preview-area');
const userInput = document.getElementById('user-input');
const loader = document.getElementById('loader');
let selectedFile = null;

function autoGrow(element) {
  element.style.height = "auto";
  element.style.height = (element.scrollHeight) + "px";
}

function handleKeyPress(e) {
  if (e.key === 'Enter' && !e.shiftKey) {
    e.preventDefault();
    sendMessage();
  }
}

function handleFileSelect() {
  const fileInput = document.getElementById('file-input');
  if (fileInput.files.length > 0) {
    selectedFile = fileInput.files[0];
    const url = URL.createObjectURL(selectedFile);
    previewArea.innerHTML = `<div class="preview-wrapper"><div class="cancel-btn"
onclick="cancelImage()"><span class="material-symbols-
outlined">close</span></div></div>`;
  }
}

function cancelImage() {
  selectedFile = null;
  previewArea.innerHTML = "";
  document.getElementById('file-input').value = "";
}

function openModal(src) {
  const modal = document.getElementById('image-modal');
  const modalImg = document.getElementById('full-image');
  modalImg.src = src;
  modal.style.display = "flex";
}

function closeModal() {
  document.getElementById('image-modal').style.display = "none";
}

function appendMessage(content, sender, type = 'text') {
  const welcome = document.getElementById('welcome-screen');
  if (welcome) welcome.remove();
}

```

```

const msgDiv = document.createElement('div');

if (type === 'image') {
  msgDiv.className = 'user-image-container';
  msgDiv.innerHTML = ``;
  chatContainer.appendChild(msgDiv);
} else {
  msgDiv.className = sender === 'user' ? 'user-message' : 'ai-
message';
  chatContainer.appendChild(msgDiv);

  if (sender === 'ai') {
    typeEffect(msgDiv, content);
  } else {
    msgDiv.innerHTML = content;
  }
}
chatContainer.scrollTop = chatContainer.scrollHeight;
}

function typeEffect(element, fullText) {
  let index = 0;
  const speed = 1;
  element.innerHTML = `<div class="typing-wrapper"></div>`;
  const wrapper = element.querySelector('.typing-wrapper');

  const timer = setInterval(() => {
    if (index < fullText.length) {
      index += 2;
      if (index > fullText.length) index = fullText.length;
      wrapper.innerHTML = marked.parse(fullText.substring(0,
index));
      chatContainer.scrollTop = chatContainer.scrollHeight;
    } else {
      clearInterval(timer);
      element.innerHTML = marked.parse(fullText);
    }
  }, speed);
}

async function sendMessage() {
  const text = userInput.value.trim();
  if (!text && !selectedFile) return;

  if (selectedFile)
appendMessage(URL.createObjectURL(selectedFile), 'user', 'image');
  if (text) appendMessage(text, 'user');
}

```

```

const fileToSend = selectedFile;
const textToSend = text;

userInput.value = "";
userInput.style.height = "auto";
cancelImage();

await processUpload(fileToSend, textToSend);
}

async function processUpload(file, textQuery) {
  loader.style.display = 'flex';
  const formData = new FormData();
  if (file) formData.append('file', file);
  // Even if query is empty, send an empty string or specific instruction
  formData.append('query', textQuery || "");

  // Send the history gathered so far
  formData.append('history', JSON.stringify(chatHistory));

  try {
    const response = await fetch('http://127.0.0.1:8000/analyze/ui-
screenshot', {
      method: 'POST',
      body: formData
    });
    const data = await response.json();

    if (response.ok) {
      // 1. Handle Visual Feedback (The image with boxes)
      if (data.audited_image_base64) {
        const imgSrc =
`data:image/jpeg;base64,${data.audited_image_base64}`;
        appendMessage(imgSrc, 'ai', 'image');
      }

      // 2. Prepare the combined text content
      let aiContent = "";
      if (data.opencv_data) {
        aiContent = `#### Technical Metrics\n* **Whitespace:**
${data.opencv_data.whitespace_ratio}%\n* **Small Elements:**
${data.opencv_data.small_elements_found}\n\n---\n\n`;
      }
      aiContent += data.gemini_response;

      // 3. Display the message to the user
      appendMessage(aiContent, 'ai');

      // 4. UPDATE HISTORY (Crucial for memory)
      // Store the text query and the RAW gemini response

```

```

        chatHistory.push({role: "user", parts: [{text: textQuery}]});
        chatHistory.push({ role: "model", parts: [{ text:
data.gemini_response }] });

        // Keep history lean (Last 10 turns = 20 entries)
        if (chatHistory.length > 20) chatHistory = chatHistory.slice(-
20);

        } else {
            appendMessage(`Error: ${data.detail || "Server error"}`, "ai");
        }
    } catch (error) {
        appendMessage("Backend is not responding. Ensure your Python
server is running.", "ai");
    } finally {
        loader.style.display = 'none';
    }
}

function toggleDropdown(event) {
    event.stopPropagation();
    document.getElementById("help-
dropdown").classList.toggle("show");
}

// Close the dropdown if the user clicks outside of it
window.onclick = function(event) {
    const dropdown = document.getElementById("help-dropdown");
    if (dropdown && dropdown.classList.contains('show')) {
        dropdown.classList.remove('show');
    }
}

document.addEventListener("paste", (event) => {
    const items = event.clipboardData?.items;
    if (!items) return;

    for (let item of items) {
        if (item.type.indexOf("image") !== -1) {
            const file = item.getAsFile();

            if (!file) return;

            // Store as selected file (same flow as upload button)
            selectedFile = file;

            // Show preview (reuse your existing preview UI)
            const url = URL.createObjectURL(file);

            previewArea.innerHTML = `

```

```

<div class="preview-wrapper">
  
  <div class="cancel-btn" onclick="cancelImage()">
    <span class="material-symbols-outlined">close</span>
  </div>
</div>
`
};
}
});
</script>
</body>
</html>

```

database.py

```
import os
from datetime import datetime, timezone
from motor.motor_asyncio import AsyncIOMotorClient
import certifi
from dotenv import load_dotenv

load_dotenv()

class AuditDatabase:
    def __init__(self):
        self.uri = os.getenv("MONGO_URI")

        # Use motor for ASYNC support
        self.client = AsyncIOMotorClient(self.uri, tlsCAFile=certifi.where())
        self.db = self.client["ui_audit_db"]
        self.collection = self.db["audits"]

    async def save_audit(self, filename, query, metrics, report):
        # Save the audit result asynchronously

        try:
            document = {
                "timestamp": datetime.now(timezone.utc),
                "filename": filename or "no_file",
                "user_query": query,
                "metrics": metrics,
                "gemini_report": report
            }
            await self.collection.insert_one(document)
            return True

        except Exception as e:
            print(f"Database Error: {e}")
            return False

# Initialize instance
db_helper = AuditDatabase()
```

```
:root {
  --bg-color: #fcfcfd;
  --user-bubble: #f0f2f5;
  --text-main: #1f2937;
  --border-color: #eeeeee;
  --hover-gray: #F5F5F5;
  --content-width: 850px;
}

* {
  box-sizing: border-box;
}

body, html {
  margin: 0;
  padding: 0;
  font-family: 'Inter', -apple-system, BlinkMacSystemFont, "Segoe UI",
  Roboto, sans-serif;
  background-color: var(--bg-color);
  overflow-y: auto;
  height: 100%;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

#app-container {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  width: 100%;
  align-items: center;
  position: relative;
}

#top-header {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  background-color: var(--bg-color);
  height: 65px;
  z-index: 1000;

  display: flex;
  justify-content: center;
  align-items: center;
}
```



```

.header-content {
  width: 100%;
  max-width: 100%;
  padding: 0 24px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.logo-text {
  font-weight: 600;
  font-size: 1.1rem;
  color: var(--text-main);
}

.dropdown {
  position: relative;
  display: inline-block;
}

/* The Dropdown Menu */
.dropdown-content {
  display: none; /* Hidden by default */
  position: absolute;
  right: 16px;
  background-color: #ffffff;
  min-width: 170px;
  box-shadow: 0px 8px 16px rgba(0,0,0,0.1);
  border: 1px solid var(--border-color);
  border-radius: 12px;
  padding: 8px 0;
  z-index: 1001;
  overflow: hidden;

  user-select: none;
}

.dropdown-content.show {
  display: block;
}

/* Individual Rows */
.dropdown-item {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 10px 16px;
  cursor: pointer;
  transition: background 0.2s;
  color: var(--text-main);
}

```

```

}

.dropdown-item:hover {
  background-color: var(--hover-gray);
}

.dropdown-item .material-symbols-outlined {
  font-size: 20px;
  /* Since we want these filled for consistency with your other icons */
  font-variation-settings: 'FILL' 1, 'wght' 400, 'GRAD' 0, 'opsz' 20;
}

/* Show class for JS */
.show {
  display: block;
}

#chat-container {
  width: 100%;
  max-width: var(--content-width);
  padding: 40px 20px 200px 20px;
  display: flex;
  flex-direction: column;
  gap: 24px;
  flex: 1;
  padding-top: 85px !important;
}

#welcome-screen {
  margin-top: 20vh;
  text-align: center;
  width: 100%;
}

#welcome-screen h1 {
  font-size: 2.8rem;
  font-weight: 600;
  color: #111827;
  margin-bottom: 12px;
}

.input-area {
  position: fixed;
  bottom: 0;
  width: 100%;
  display: flex;
  flex-direction: column;
  align-items: center;
  background: linear-gradient(to top, var(--bg-color) 70%, transparent);
  padding: 10px 0 20px 0;
}

```

```
    z-index: 100;
  }

.input-container {
  width: 90%;
  max-width: var(--content-width);
  max-height: 250px;
  background: #ffffff;
  border: 1px solid #e5e5e5;
  border-radius: 24px;
  padding: 10px;
  display: flex;
  flex-direction: column;
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.05);
}

.disclaimer-text {
  font-size: 12px;
  color: #9ca3af;
  margin-top: 12px;
  text-align: center;
  width: 100%;
  letter-spacing: 0.1px;
}

#user-input {
  width: 100%;
  border: none;
  outline: none;
  font-size: 16px;
  resize: none;
  padding: 12px;
  background: transparent;
  font-family: inherit;
}

#preview-area {
  display: flex;
  padding: 10px;
  gap: 10px;
}

.preview-wrapper {
  position: relative;
  display: inline-block;
}

.pending-img {
  height: 80px;
  width: 80px;
}
```

```

    object-fit: cover;
    border-radius: 12px;
    border: 1px solid var(--border-color);
  }

.cancel-btn {
  position: absolute;
  top: -5px;
  right: -5px;
  background: #1f2937;
  color: white;
  border-radius: 50%;
  width: 22px;
  height: 22px;
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  border: 2px solid white;
  box-shadow: 0 2px 4px rgba(0,0,0,0.2);
}

.cancel-btn span {
  font-size: 14px !important;
}

.user-message {
  align-self: flex-end;
  background-color: var(--user-bubble);
  color: var(--text-main);
  padding: 12px 20px;
  border-radius: 20px;
  max-width: 80%;
  line-height: 1.6;
}

.user-image-container {
  align-self: flex-end;
  margin-bottom: 5px;
}

.image-preview {
  max-width: 350px;
  max-height: 450px;
  object-fit: contain;
  border-radius: 16px;
  border: 1px solid var(--border-color);
  box-shadow: 0 4px 12px rgba(0,0,0,0.08);
}

```

```

.ai-message {
  width: 100%;
  min-width: 0;
  line-height: 1.8;
  color: #374151;
  font-size: 16px;
  text-align: justify;
  margin-top: 10px;
}

.typing-wrapper {
  width: 100%;
}

/* --- Buttons & Icons --- */
.input-actions {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 5px;
}

.icon-btn {
  background: none;
  border: none;
  cursor: pointer;
  color: #4b5563;
  display: flex;
  align-items: center;
  justify-content: center;
  width: 36px;
  height: 36px;
  border-radius: 50%;
  transition: 0.2s;
}

.icon-btn:hover {
  background-color: var(--hover-gray);
  color: #111827;
}

#loader {
  position: fixed;
  bottom: 160px;
  left: 50%;
  transform: translateX(-50%);
  font-size: 14px;
  color: #6b7280;
  background: #ffffff;
  padding: 8px 20px;
}

```

```

border-radius: 20px;
border: 1px solid var(--border-color);
z-index: 2000;
box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
display: none;
align-items: center;
gap: 8px;
}

body::-webkit-scrollbar {
width: 8px;
}

body::-webkit-scrollbar-thumb {
background: #d1d5db;
border-radius: 10px;
}

body::-webkit-scrollbar-track {
background: transparent;
}

.input-area,
.input-container,
.icon-btn,
.cancel-btn,
#welcome-screen,
.user-image-container {
-webkit-user-select: none; /* Safari */
-ms-user-select: none; /* IE 10 and 11 */
user-select: none; /* Standard syntax */
}

.ai-message,
.user-message,
#user-input {
-webkit-user-select: text;
-ms-user-select: text;
user-select: text;
}

/* Remove the blue "tap highlight" on mobile devices for buttons */
.icon-btn, .cancel-btn {
-webkit-tap-highlight-color: transparent;
}

.modal {
display: none; /* Keep this as default */
position: fixed;
z-index: 9999; /* Higher z-index to ensure it is on top of everything */

```

```
left: 0;
top: 0;
width: 100%;
height: 100%;
background-color: rgba(0, 0, 0, 0.8);
align-items: center;
justify-content: center;
}

.modal-content {
  max-width: 80%; /* Set to your 80% preference */
  max-height: 80%;
  border-radius: 12px;
  background-color: white;
  box-shadow: 0 20px 25px -5px rgba(0, 0, 0, 0.2);
  cursor: default;
}

.image-preview {
  cursor: pointer;
  transition: opacity 0.2s;
}
.image-preview:hover {
  opacity: 0.9;
}
```

8.4. Proposal Report



SEGi
University &
Colleges



College Name	University of Hertfordshire SEGi College Subang Jaya											
Programme Name	BACHELOR OF SCIENCE (HONS) COMPUTER SCIENCE (ARTIFICIAL INTELLIGENCE)											
Module Name	Artificial Intelligence Project	Module Code	6COM2017									
		Semester	Sept 2025									
Module Leader	Ms Anizah Abu Bakar	Assessment Type	Proposal Report									
Lecturer Name	Mr. Arriff Jalal											
Student's declaration	I hereby certify that this assignment is my own work and where materials have been used from other resources, they have been properly acknowledged. I also understand I will face the possibility of failing the module if the content of this assignment is plagiarized. <table border="1"> <thead> <tr> <th>No.</th> <th>Name</th> <th>Student ID</th> <th>Signature</th> </tr> </thead> <tbody> <tr> <td>1</td> <td>Kenneth Chong Vui Kin</td> <td>SCSJ2100435</td> <td></td> </tr> </tbody> </table>				No.	Name	Student ID	Signature	1	Kenneth Chong Vui Kin	SCSJ2100435	
No.	Name	Student ID	Signature									
1	Kenneth Chong Vui Kin	SCSJ2100435										
Release Date		Submission Due Date	31/10/2025	Marks Obtained: 								
Date Received		Student's work assessed by / date										

Table of Content

Table of Content	112
List of Figures	113
List of Tables	114
1. Introduction	115
2. Background	117
3. Problem Statement	118
4. Literature Review	119
5. Research Questions and Objectives	122
5.1. Research Questions	122
5.2. Objectives	123
6. Proposed Methodology	124
6.1. Methodology	124
6.2. Gantt Chart	126
7. Project Significance	130
8. Scope and Limitations	131
8.1. Scope	131
8.2. Limitations	132
9. Expected Outcome	133
10. References	134

List of Figures

Figure 6.1.1 Agile Methodology	124
Figure 6.2.1 Proposed Project Timeline.....	126
Figure 6.2.2 Project Phase 1 - Planning Phase.....	127
Figure 6.2.3 Project Phase 2 - Design Phase	127
Figure 6.2.4 Project Phase 3 - Development Phase	128
Figure 6.2.5 Project Phase 4 - Testing Phase	128
Figure 6.2.6 Project Phase 5 & 6 - Deployment & Documentation Phase	128

List of Tables

Table 5.1.1 Phases of Agile Methodology	125
---	-----

1. Introduction

The quality of User Interface (UI) and User experience (UX) has become a key factor in the success of modern digital products. UI/UX affects user satisfaction and engagement. However, the design process often faces significant challenges. Manually conducting comprehensive heuristic evaluations is time-consuming and can be inconsistent. This inconsistency arises from the subjective nature and varied expertise of human evaluators. As a result, this bottleneck delays critical feedback, leading to late discovery of usability issues, which is expensive and difficult to remediate during later development stages.

To address the need for accessible, this project proposes the development of an “AI Chatbot for UI Improvement”. This application will help UI/UX designers and developers to identify common user interface (UI) and user experience (UX) issues. The core of the system is a hybrid-logic chatbot that analyzes uploaded UI screenshots with 2 approaches:

- Rule-based Logic: Uses Python implementation to quickly identify basic and common UI issues based on predefined heuristics.
- Advanced AI Analysis: Uses Gemini 1.5 flash to analyze the visual data in the screenshot and provide more heuristic feedback and suggestions for improvement.

The application will be developed with Python-centric stack for prototyping and development. The backend will use FastAPI in Python to enable fast communication. Screenshot uploads are handled through a standard File upload interface, while the backend uses OpenCV to process the images for analysis. All data will be managed efficiently using MongoDB Atlas Database. Finally, the frontend will be developed using standard web technologies – HTML, CSS and JavaScript to deliver responsive chatbot interface.

The development of the “AI Chatbot for UI Improvement” supports several United Nations Sustainable Development Goals (SDGs) by promoting efficiency and access to quality tools.

One of the SGD is SDG 9: Industry, Innovation and Infrastructure. By automating the design review process, the tool directly supports innovation and helps build reliable infrastructure such as digital products. It improves their quality and makes them less likely to need expensive fixes. This leads to better productivity in the digital industry.

2. Background

User Interface (UI) design refers to the visual and interactive elements of a product such as buttons and icons of a website. The goal of UI design is to make a product pleasing to users and easy to use. On the other hand, User Experience (UX) refers to how the user feel during the interaction with the product. It focuses on the feel, flow and overall usability and ensures the product is enjoyable to use.

UI and UX is increasingly becoming important in today's era. A well-designed interface can significantly enhance user satisfaction and engagement. Conversely, a poor UI/UX design can lead to frustration and increased cognitive load. Studies have shown that a positive user experience is directly linked to a company's success. According to Luther et al., 2020, UX has become a core element to the success of digital products and businesses. Companies that implement strong user experience practices increase user satisfaction and loyalty.

Artificial Intelligence (AI) is transforming the field of UI/UX by automating repetitive tasks. AI enhances user experience through personalization, recommendation engines and intelligent assistance (Virvou, 2023). A systematic literature review shows that AI is increasing used across the UX design process, from gathering requirements to design and evaluation (Stige et al., 2023). Furthermore, AI can act as a creative partner for designers, which supports brainstorming (Khan et al., 2025).

3. Problem Statement

UI/UX designers those work on small teams or as individual freelancers usually face significant challenges during the design process. Manually conducting a thorough heuristic evaluation is time-consuming. This can lead to differences in findings among different evaluators (Göransson, 2004). Although the process is important, it requires a lot of expertise and is difficult to standardize, which can result in various outcomes (Huang & Benyoucef, 2022). Furthermore, not having immediate feedback can result in usability issues that are only found during the development process, which raises the costs of fixing the issues (Boehm & Basili, 2005). This phenomenon is also known as “usability debt,” can accumulate and become a burden on a project. It can delay launch and require additional work to resolve (Cai & Kazman, 2023).

One significant challenge is the complexity of modern applications. Manually reviewing every screen and interaction is impractical, which raises the risk of creating usability debt (Meniar et al., 2017). The problem is especially true for mobile interfaces where many design guidelines are important but hard to apply consistently across large projects (Ballantyne et al., 2018). Dependence on a limited number of human evaluators who have their own biases and blind spots creates a bottleneck that slows down feedback. As a result, common issues such as unclear system status or insufficient error prevention go unnoticed (Laokitpanich & Senivongse, 2022). These issues highlight the need for a tool that can automate and streamline the design review process, then provide timely feedback to ensure high-quality user experience.

4. Literature Review

According to Gada, T. & Chudasama, S. (2024) User-centric approach and other methods are crucial in product design to improve user experience. Combining data analysis and business intelligence helps in making informed decisions. These informed decisions ensure the products are functional and satisfy users.

Padmasiri *et al.*, (2023) The paper studies the impact of AI on User Experience (UX) design. AI has the potential to create user experiences that satisfied users. However, there are still challenges when integrating AI. The study offers a guide for UX professionals to overcome the challenges and optimize AI use during the design process.

Khan, A. *et al.* (2025) The study explores how AI tools affect the divergent thinking phases of UI/UX design, especially in research or ideation. The paper shows that designers value AI for giving them more control over ideation and help them to unlock their creativity. The research also highlights how AI supports creativity and generates alternatives.

Lecaros, A. *et al.* (2021) The paper studies the challenges and opportunities in automating heuristic evaluations. The paper identifies many challenges such as the limited applicability of Nielsen heuristics, the low expertise of evaluators and the lack of formalization during the evaluations. It also shows that there are limited software solutions that are able to automate the evaluations. The paper suggests that developing a tool can address the stated issues and provide a more efficient evaluation process.

Takaffoli, M. *et al.* (2024) The study shows the challenges of adopting Generative Artificial Intelligence (GenAI) tools in the User Experience field. The paper discovers a lack of company policies and team-wide practices for the GenAI use. It also calls for enhanced training to improve the effectiveness of prompts and output evaluations. The paper suggests that enterprises to build clear policies and provide relevant trainings. The

study emphasizes designers still face limitations when performing more complex tasks although the researches can benefit from GenAI.

Wu, J. *et al.* (2024) The paper provides a detailed survey of visual prompting methods in multimodal large language models (MLLMs). It highlights techniques for visual prompting, prompt generation and composition reasoning. The paper examines how visual prompts improve MLLM alignment, perception and understanding. It also reviews the training methods and suggests future research directions to improve visual reasoning in MLLMs.

Rotaru, O.A. *et al.* (2020) The paper analyzes commonly used standardized questionnaires for usability evaluation such as SUS, PSSUQ, QUIS and SUMI. A case study with students compared these surveys for evaluating projects. SUS has the highest scores at 60% compared to others. The paper outlines the advantages and disadvantages of each questionnaire and suggests further work that involves using these questionnaires.

Xiong, H. *et al.* (2024) The paper studies the convergence of Symbolic AI (logic-based rules) and Connectionist AI (LLMs) can form the powerful paradigm of Neuro-Symbolic AI. This synergy enables improved reasoning and decision-making by utilizing LLMs to perform logical tasks. The principle directly validates the project's hybrid-logic chatbot, which combines rule-based heuristics with the capabilities of Gemini 1.5 Flash for analysis.

Iniesto, F. *et al.* (2022) The paper shows that Conversational User Interfaces (CUIs) effectively reduce the burden of complex processes. Users prefer them over static forms for accessing technical support. This shows that CUIs can provide analytical feedback in a more user-friendly way. The research highlights the needs to design accessible CUIs carefully.

Chen, X. *et al.* (2024) This paper shows that effective computer vision relies on solving low-level tasks with frameworks like Visual Task Prompt-based Image processing (VPIP).

This framework handles different tasks using visual prompts. This approach supports the use of OpenCV for initial image parsing and extraction.

5. Research Questions and Objectives

5.1. Research Questions

The goal of this project is to overcome the limitations of manual heuristic evaluation by providing fast and accessible analysis. Below are the five research questions that will guide the project's development:

6. How can a hybrid AI system balance objective consistency (through rule-based logic) with nuanced insights (through Gemini 1.5 flash) to generate actionable UI/UX improvement suggestions?
 - This question focuses on the core fusion methodology, investigating how to combine the output of Python rules with generative LLM to deliver the analysis.
7. What is the optimal technical pipeline required to process UI screenshot and transform it into a structured data format that maximizes the quality of the analysis output?
 - This question addresses the backend flow, focusing on file handling, image parsing using OpenCV within the FastAPI to ensure optimal performance.
8. How can the output of the rule-based logic be structured and integrated into the prompt to guide the Gemini 1.5 flash model to provide relevant feedback?
 - This question targets the prompt challenge, focusing on how the rule-based findings should be formatted to serve as reliable context for the LLM to enhance the precision of the analysis.
9. How does AI chatbots reduced “usability debt” bottleneck by providing actionable feedback compared to time-consuming manual evaluation?

- This question relates to the project's value. It aims to accelerate the design review process and deliver timely feedback for the critical UI/UX issues.

10. What criteria must be implemented in the frontend design to ensure the AI chatbot provides responsive user experience when displaying complex analysis?

- This question focuses on the usability aspect. It looks into the best practices in designing the chat interface to present feedback and maintain user trust.

5.2. Objectives

The objectives are:

- To allow users to upload UI screenshots or describe a UI screen and receive usability improvement suggestions
- To process the input using a rule-based engine that detects common UI/UX issues
- To provide enhanced UI improvement suggestions by integrating lightweight AI model

6. Proposed Methodology

6.1. Methodology



Figure 6.1.1 Agile Methodology

The development of the “AI Chatbot for UI Improvement” will use agile development methodology. This approach is popular for its adaptability and flexibility, making it ideal for projects with complex iterations such as combining a rule-based system with the Google Gemini AI Model and evolving requirements. Agile methodology focuses on delivering the product in small steps rather than one big final result. This project will be broken down into short cycles. The core functions are built and tested in sequence to ensure a testable version of the project is available. The strength of the agile approach focuses on continuous feedback and adjustment. Regular meetings with the supervisor provide feedback, allowing for changes and updates problems arise. There are several phases in the agile methodology:

Phases	Description
--------	-------------

Planning	This phase involves defining the overall project title and the objectives, then a proposal is made. Problem statements and objectives are researched and features are prioritized to achieve the objectives
Design	This phase involves designing the core components for the project such as Backend using FastAPI, Database using MongoDB etc. UI sketches for the project are also designed
Development	This phase focuses on coding and implementing the techniques mentioned earlier in the design phase, where the designs are being transformed into usable and testable project
Testing	Testing phase is integrated after development during each cycle. This includes testing the core logic, integration and usability
Deployment	As the functional increments are completed, these increments can be prepared for the final online deployment to demonstrate real-world accessibility
Review and Documentation	This phase involves demonstrating the working features completed during the cycle. Feedback from the supervisor is collected and fed back into the next planning phase for maintenance. Final report Submission and project presentation is also involved

Table 6.1.1 Phases of Agile Methodology

6.2. Gantt Chart

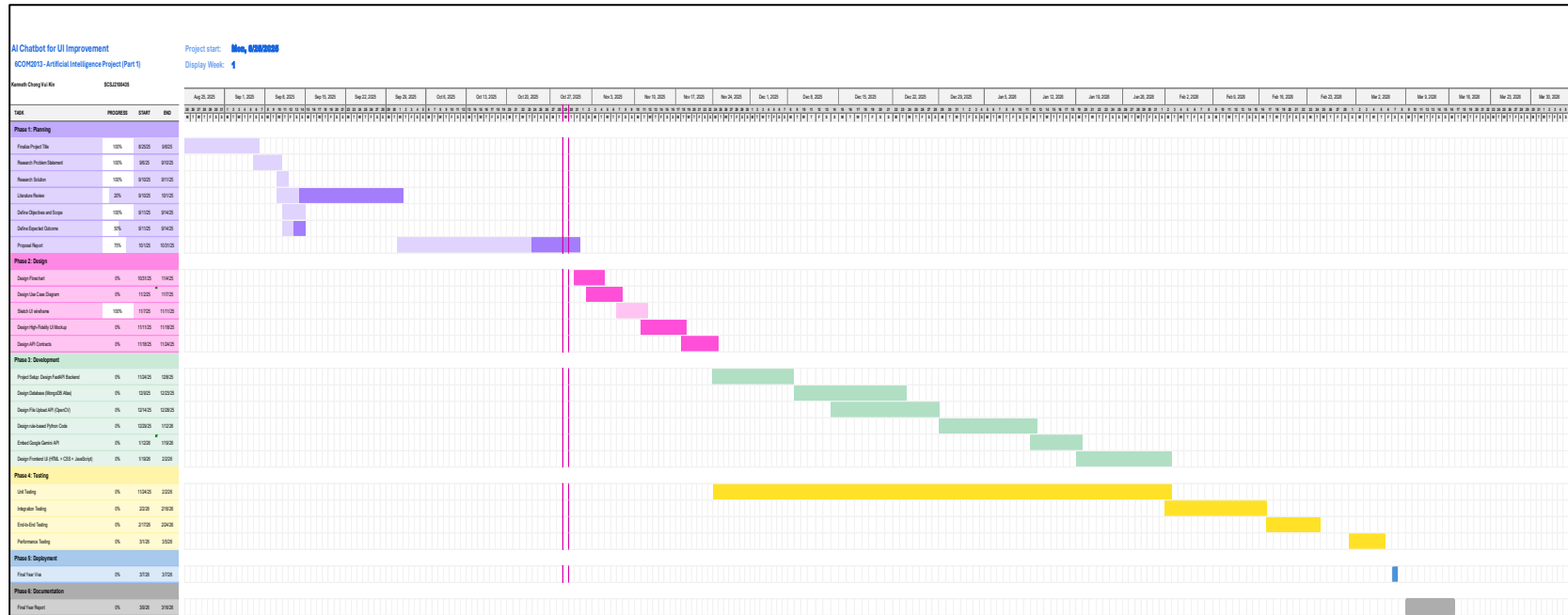


Figure 6.2.1 Proposed Project Timeline

Above is the Gantt Chart as a guideline for the project. The project has different phases. The planning phase was started at 25 August 2025. At the planning phase, project title was set after the discussion with supervisor, other planning and research are started after the title was set.

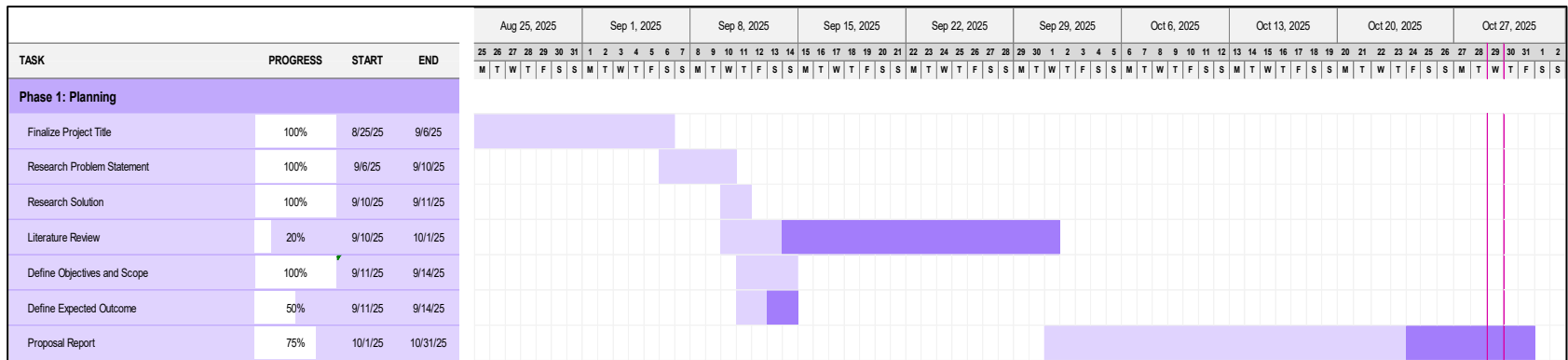


Figure 6.2.2 Project Phase 1 - Planning Phase

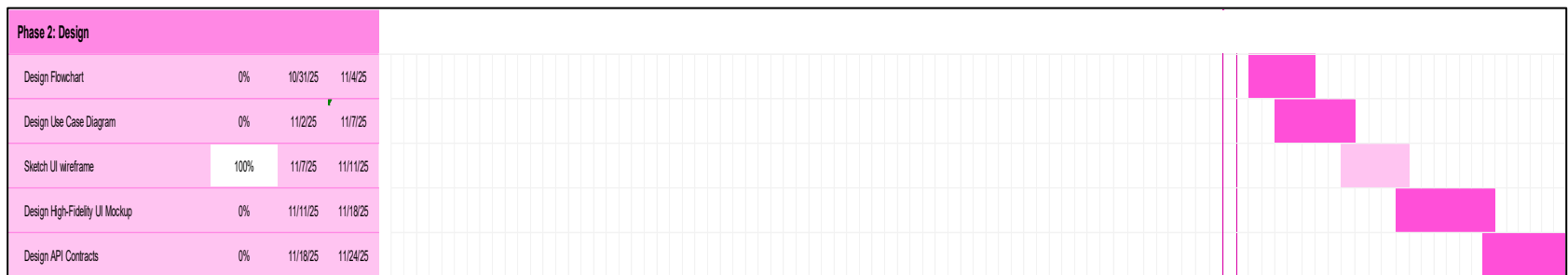


Figure 6.2.3 Project Phase 2 - Design Phase

The design phase is to define and design the architecture by defining and designing use case diagram, system flowchart. Then, low-fidelity and high-fidelity wireframes.

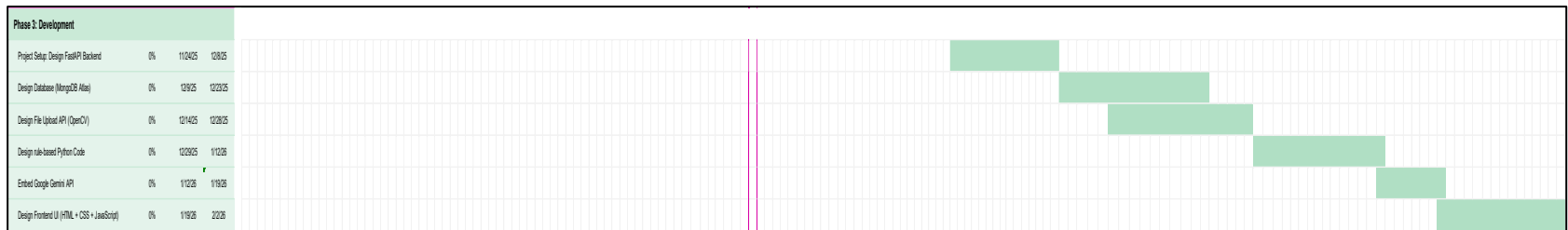


Figure 6.2.4 Project Phase 3 - Development Phase

This is the phase where the designed architecture, interfaces and system logics are being turned into a testable and runnable program.

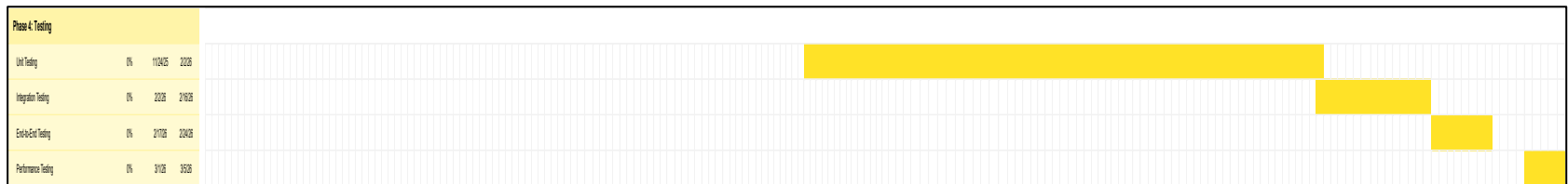


Figure 6.2.5 Project Phase 4 - Testing Phase

Unit testing starts right after a function is done implemented to ensure bugs or mistakes can be identified and solved in early stages. After all functions are done testing, integration testing is performed so that the functions can work together smoothly.

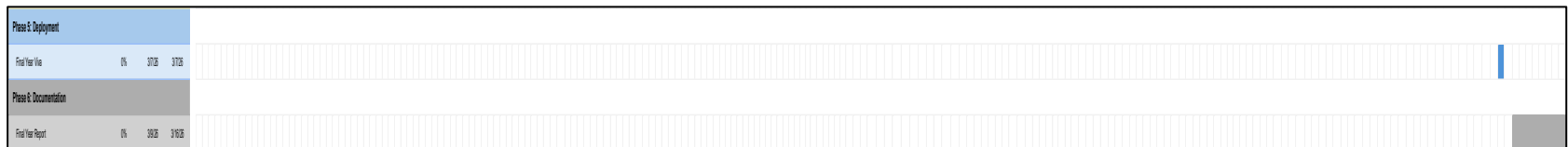


Figure 6.2.6 Project Phase 5 & 6 - Deployment & Documentation Phase

7. Project Significance

This project provides a straightforward, accessible solution to common problems in UI/UX design, making it significant for both practical use and demonstrating modern AI capabilities.

The main goal is to give designers an accessible tool to improve their work by offering instant and actionable feedback. Getting detailed feedback is often slow especially for people working alone or in smaller teams. The AI chatbot helps user to catch common UI/UX issues earlier to avoid the mistakes. This saves designers time and prevents the high cost of fixing major problems later on during development. This tool is useful for start-ups, small businesses and students because it offers design review without the needing to hire expert consultants to help them build digital products.

This project serves as a proof-of-concept for a smart new way to use AI in design. It demonstrates how the Advanced AI can act as a partner in the creative process. It provides human-like suggestions on design principles and aesthetics. By combining this powerful AI with simple Python-based rules. The system ensures that the feedback is not only smart but also reliable.

8. Scope and Limitations

This section defines the boundaries of the “AI Chatbot for UI Improvement” project, outlining the specific features and functionalities that will be developed, as well as the constraints and simplifying assumptions made due to time, resources and complexity.

8.1. Scope

The scope of this project is defined by the objectives and focuses on creating a product that demonstrates the hybrid AI analysis methodology. The core functionality is the system will accept a single UI screenshot and return an analysis report.

There are 2 analysis approach for the hybrid analysis implementation: Rule-based Logic (Python) and Advanced AI analysis. Rule-based Logic focus on objective checks such as identifying low colour contrast, basic component alignment and other UI elements. Advanced AI analysis focus on providing feedback on visual hierarchy.

The project is scoped to the defined stack: FastAPI backend, MongoDB Atlas for data persistence and a basic HTML, CSS, JavaScript frontend for a single-page chatbot interface.

8.2. Limitations

There are certain complexities and limitations outside the current scope:

- **Analysis Depth** – The system is limited to analyze a single screenshot at a time. It will not analyze multi-screen user flows and interactive states such as hover animations.
- **User Authentication** – The system will focus on core analysis and will not implement user accounts and logins.
- **Real-time Data** – While the system is embedded with Google Gemini API, the performance are limited by the free tier access to the model are not guaranteed for production-level scale.

9. Expected Outcome

This project is expected to yield three outcomes: a hybrid analysis tools and a positive impact on the early stages of UI/UX design workflow.

The first outcome is the delivery of a functional hybrid UI Analysis Chatbot. The developed application will provide a seamless interface for users to upload a UI screenshot and receive an analysis. The system will generate feedback for the user. This outcome fulfils the project's objectives.

The second outcome is the project's positive impact and demonstration value in the UI/UX design process. This tool addresses the project's problem statement by significantly reducing the time and cost associated with manual heuristic evaluations, thereby helping designers to reduce the time needed. This project will serve as a Proof-of-Concept for the analysis.

10. References

- Luther, L., Tiberius, V., & Brem, A. (2020). User Experience (UX) in Business, Management, and Psychology: A bibliometric mapping of the current state of research. *Multimodal Technologies and Interaction*, 4(2), 18. <https://doi.org/10.3390/mti4020018>
- Virvou, M. (2023). Artificial Intelligence and User Experience in reciprocity: Contributions and state of the art. *Intelligent Decision Technologies*, 17(1), 73–125. <https://doi.org/10.3233/idt-230092>
- Stige, Å., Zamani, E. D., Mikalef, P., & Zhu, Y. (2023). Artificial intelligence (AI) for user experience (UX) design: a systematic literature review and future research agenda. *Information Technology and People*, 37(6), 2324–2352. <https://doi.org/10.1108/itp-07-2022-0519>
- Khan, A., Shokrizadeh, A., & Cheng, J. (2025). Beyond Automation: How UI/UX Designers Perceive AI as a Creative Partner in the Divergent Thinking Stages. *ArXiv (Cornell University)*. <https://doi.org/10.1145/3706598.3713500>
- Bengt Göransson. (2004, June). *User-Centred Systems Design - Designing Usable Interactive Systems in Practice*. ResearchGate; unknown. https://www.researchgate.net/publication/266897135_User-Centred_Systems_Design_-_Designing_Usable_Interactive_Systems_in_Practice
- Huang, Z., & Benyoucef, M. (2022). An Empirical Study of Mobile Application Usability: A Unified Hierarchical approach. *International Journal of Human-Computer Interaction*, 39(13), 2624–2643. <https://doi.org/10.1080/10447318.2022.2082021>
- Boehm, B., & Basili, V. (2005). Software defect Reduction Top-10 list. In *Springer eBooks* (pp. 426–431). https://doi.org/10.1007/3-540-27662-9_26

Cai, Y., & Kazman, R. (2023). Software design analysis and technical debt management based on design rule theory. *Information and Software Technology*, 164, 107322. <https://doi.org/10.1016/j.infsof.2023.107322>

Meniar, C., Opalvens, F., & Hallé, S. (2017). Runtime verification of user interface guidelines in mobile devices. In *Lecture notes in computer science* (pp. 410–415). https://doi.org/10.1007/978-3-319-67531-2_27

Ballantyne, M., Jha, A., Jacobsen, A., Hawker, J. S., & El-Glaly, Y. N. (2018). Study of Accessibility Guidelines of Mobile Applications. *Proceedings of the 17th International Conference on Mobile and Ubiquitous Multimedia - MUM 2018*. <https://doi.org/10.1145/3282894.3282921>

Laokitpanich, J., & Senivongse, T. (2022). *Heuristic-Based Usability Evaluation Tool for Android Applications for Senior Citizens*, 83–88. <https://doi.org/10.1145/3578997.3579011>

Gada, T. & Chudasama, S. (2024). The Role of User Experience in Effective Product Design Exercises Strategies for Incorporating User-Centric Approaches and Data Analysis with Business Intelligence. *International Research Journal of Modernization in Engineering Technology and Science*. <https://doi.org/10.56726/irjmets56245>

Padmasiri, P. *et al.* (2023) 'AI-Driven User Experience Design: Exploring Innovations and Challenges in Delivering Tailored User Experiences,' *2023 8th International Conference on Information Technology Research (ICITR)*, pp. 1–6. <https://doi.org/10.1109/icitr61062.2023.10382802>.

Khan, A. *et al.* (2025) 'Beyond Automation: How Designers Perceive AI as a Creative Partner in the Divergent Thinking Stages of UI/UX Design,' *CHI '25: Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pp. 1–12. <https://doi.org/10.1145/3706598.3713500>.

Lecaros, A. *et al.* (2021) 'Challenges and Opportunities on the Application of Heuristic Evaluations: A Systematic Literature review,' in *Lecture notes in computer science*, pp. 242–261. https://doi.org/10.1007/978-3-030-78221-4_17.

Takaffoli, M. *et al.* (2024) 'Generative AI in User Experience design and Research: How do UX practitioners, teams, and companies use GenAI in industry?,' *Designing Interactive Systems Conference*, pp. 1579–1593. <https://doi.org/10.1145/3643834.3660720>.

Wu, J. *et al.* (2024) 'Visual prompting in multimodal large Language models: a survey,' *arXiv (Cornell University)* [Preprint]. <https://doi.org/10.48550/arxiv.2409.15310>.

Rotaru, O.A. *et al.* (2020) 'Standardised questionnaires in usability evaluation. Applying standardised usability questionnaires in digital products evaluation,' in *Communications in computer and information science*, pp. 39–48. https://doi.org/10.1007/978-3-030-59506-7_4.

Xiong, H. *et al.* (2024) 'Converging Paradigms: The synergy of symbolic and connectionist AI in LLM-Empowered Autonomous agents,' *arXiv (Cornell University)* [Preprint]. <https://doi.org/10.48550/arxiv.2407.08516>.

Iniesto, F. *et al.* (2022) 'Creating ‘a Simple Conversation’: Designing a Conversational User Interface to Improve the Experience of Accessing Support for Study,' *ACM Transactions on Accessible Computing*, 16(1), pp. 1–29. <https://doi.org/10.1145/3568166>.

Chen, X. *et al.* (2024) 'Learning A Low-Level Vision Generalist via Visual Task Prompt,' *MM '24: Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 2671–2680. <https://doi.org/10.1145/3664647.3681621>.